

INM7062: Deep Reinforcement Learning Hospital Bed Allocation

Module Leaders: Dr. Eduardo Alonso, Dr. Atif Riaz

Group: Patrick Maw and Yehoshua Perez Condori

GitHub Repository:

http://www.github.com/YBenjaminPCondori/Clinical_Patient_Scheduling

Contents

Part A: Basic - Tabular Q-Learning	2
1. Environment and Problem Definition	2
2. State Transition and Reward Functions	3
State space $\mathcal{S}$	3
Action space $\mathcal{A}$	3
Transition function	3
Reward function	4
Emergency rejection metric	5
ICU utilisation metric	5
3. Q-Learning Setup	7
Q-table	7
Parameters	7
Q-learning update	8
4. Q-Learning Performance	8
Running Q-learning and measuring performance	8
Q-learning performance interpretation	8
Parameter and Policy Experiments	10
Analysis of Q-Learning Results	10
Part B: Advanced - Deep Q-Network (Team)	11
7. DQN with Two Improvements	11
8. Analysis of DQN Results	13
State space	13
Action space	13
Reward function	13
Transition dynamics	14
Part C: Individual - Advanced DRL Algorithm	15
9. Proximal Policy Optimization	15
10. PPO Results and Analysis	16
Contribution Summary	20
GenAI Use Statement	20

Part A: Basic - Tabular Q-Learning

1. Environment and Problem Definition

The environment chosen is that of hospital bed management. This is well suited to reinforcement learning because it is a sequential, high stakes decision making process with uncertainty, delayed consequences and complex trade-offs. This is exactly the conditions that reinforcement learning is designed for.

A core reason is that hospital bed management behaves like a Markov Decision Process (MDP). Decisions need to be made repeatedly as new patients arrive, beds are freed up and conditions change.

Hospitals experience continuous fluctuations in patient arrivals, discharges and acuity levels. NHS England describes bed management as similar to “Tetris or Chess”, where each placement, and its corresponding actions affects future possibilities and constraints, identical to cause and effect. Within a Hospital there are various many departments, specifically Wards and Type of Patients. Staff are often overwhelmed with options because the number of possible allocations can be huge.

Patient arrivals, deterioration, length of stay and discharge times are uncertain. Delayed consequences with bed allocation can affect many aspects of a hospital’s operation. Future emergency capacity, patient flow later in the day and the risk of bottlenecks are a few of the many consequences.

There are many competing objectives such as reducing patient moves, improving patient experience, reducing length of stay, maintaining ICU buffer capacity and avoiding emergency rejections. These are good examples of the complexity within hospital operations.

All of these make reinforcement learning a good fit for hospital bed management. For Q-learning this will be a simplified setup, with additional complexity added later to demonstrate the ability of Deep Q-Networks.

The environment will consist of a number of different ward types which need to be matched to different patient types:

- ICU beds free: $I_t \in \{0, 1, 2\}$
- General beds free: $G_t \in \{0, 1, 2, 3\}$
- Incoming patient type: $P_t \in \{emergency, urgent, elective/planned\}$

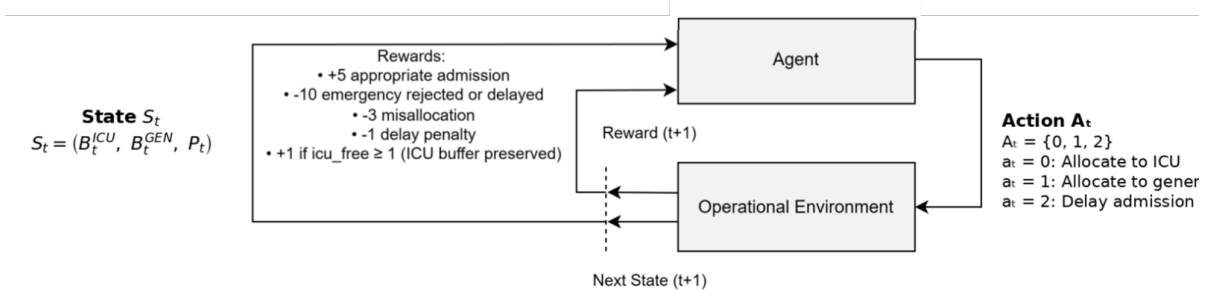


Figure A – Hospital bed-allocation MDP showing the agent, environment, state, action and reward feedback loop.

2. State Transition and Reward Functions

State space $\mathcal{S}$

Use a compact, fully discrete state:

- ICU beds free: $I_t \in \{0, 1, 2\}$
- General beds free: $G_t \in \{0, 1, 2, 3\}$
- Incoming patient type: $P_t \in \{emergency, urgent, elective/planned\}$

So a state is represented as:

$$S_t = (I_t, G_t, P_t)$$

Action space $\mathcal{A}$

Use 3 discrete actions:

- $a_t = 0$: Allocate to ICU
- $a_t = 1$: Allocate to general ward
- $a_t = 2$: Delay admission

Transition function

The transition function is kept as a step-by-step model of the original simulation logic:

- If $A_t = 0$ (allocate to ICU):
- If $I_t > 0$: $I'_t = I_t - 1$ and the patient is admitted.

Else: the patient is effectively rejected or treated as delayed with a penalty.

- If $A_t = 1$ (allocate to general ward):
- If $G_t > 0$: $G'_t = G_t - 1$ and the patient is admitted.

Else: the patient is rejected or delayed.

If $A_t = 2$ (delay):

- Beds are unchanged.
- The patient remains in a conceptual queue, modelled through a waiting penalty rather than an explicit queue.

Then:

- Discharges: at each step, with some probability, beds become free.
- For example, each occupied ICU bed has probability of being freed; the same applies for general beds.

- In a simple model:
- With probability 0.3, $I_{t+1} = \min(I'_t + 1, 2)$.
- With probability 0.4, $G_{t+1} = \min(G'_t + 1, 3)$.
- New patient arrival: sample next P_{t+1} from a categorical distribution:
- $\Pr(P_{t+1} = \textit{emergency}) = 0.2$
- $\Pr(P_{t+1} = \textit{urgent}) = 0.3$
- $\Pr(P_{t+1} = \textit{elective}) = 0.5$

So the environment step is:

1. Apply action to current patient and beds.
2. Sample discharges and update I_t and G_t .
3. Sample next patient type P_{t+1} .
4. Compute reward.
5. Return the next state, reward and episode status.

Reward function

The reward function is tied directly to clinical and operational logic:

Base rewards:

- +5 if a patient is admitted to an appropriate bed.
- Emergency/urgent $\rightarrow$ ICU or general, depending on the defined clinical appropriateness.
- Elective $\rightarrow$ general.

Penalties:

- -10 if an emergency patient is effectively rejected, for example if no bed is available and the action tries to admit, or if delay is chosen.
- -3 if misallocated.

Emergency/urgent sent to general when ICU is available, or lower-than-needed acuity allocation.

- -1 for each delayed step for any patient, either by tracking a simple waiting-time counter or by directly penalising action 2.

Optional bonus:

- +1 if $I_{t+1} \geq 1$ at the end of the step, preserving ICU buffer capacity.
- Example reward logic per step:

If $P_t = \textit{emergency}$:

- $a_t = 0$ (ICU) and $I_t > 0 \rightarrow +5$.

- $a_t = 1$ (general) and $G_t > 0$: if $I_t > 0 \rightarrow -3$ for misallocation; else $\rightarrow +2$ as better than nothing.
- $a_t = 2$ (delay) or no bed available $\rightarrow -10$.

If $P_t = \text{elective}$:

- $a_t = 1$ (general) and $G_t > 0 \rightarrow +5$.
- $a_t = 0$ (ICU) and $I_t > 0 \rightarrow -3$ for wasting ICU capacity.
- $a_t = 2 \rightarrow -1$ waiting penalty.

Emergency rejection metric

Emergency rejection is triggered when:

$$E_t = \begin{cases} 1, & P_t = \text{emergency} \wedge \text{rejected/delayed} \\ 0, & \text{otherwise} \end{cases}$$

This metric is crucial for evaluating patient safety.

ICU utilisation metric

ICU occupancy is recorded as:

$$U_t = \begin{cases} 1, & I_t < 2 \\ 0, & I_t = 2 \end{cases}$$

This provides a per-step utilisation signal.

The implemented reward function is defined over the state $S_t = (I_t, G_t, P_t)$, where I_t is the number of free ICU beds, G_t is the number of free general beds and $P_t \in \{0, 1, 2\}$ represents emergency, urgent and elective patients respectively. The action set is $A_t \in \{0, 1, 2\}$, corresponding to ICU allocation, general-bed allocation and delay.

$$R(S_t, A_t) = \begin{cases} +5, & A_t = 0, I_t > 0, P_t \in \{0, 1\}, \\ -3, & A_t = 0, I_t > 0, P_t = 2, \\ -10, & A_t = 0, I_t = 0, \\ +5, & A_t = 1, G_t > 0, P_t = 2, \\ -3, & A_t = 1, G_t > 0, P_t \in \{0, 1\}, I_t > 0, \\ +2, & A_t = 1, G_t > 0, P_t \in \{0, 1\}, I_t = 0, \\ -10, & A_t = 1, G_t = 0, \\ -1, & A_t = 2, P_t \in \{1, 2\}, \\ -11, & A_t = 2, P_t = 0. \end{cases}$$

Table 1: Implemented reward function for hospital bed allocation.

Scenario	Reward
Allocate ICU to emergency or urgent patient	+5
Allocate ICU to elective patient	-3
Attempt ICU allocation when no ICU bed is free	-10
Allocate general bed to elective patient	+5
Allocate general bed to emergency/urgent patient while ICU is available	-3
Allocate general bed to emergency/urgent patient when ICU is full	+2
Attempt general-bed allocation when no general bed is free	-10
Delay urgent or elective patient	-1
Delay emergency patient	-11

The emergency rejection indicator is tracked separately from the scalar reward:

$$E(S_t, A_t) = \begin{cases} 1, & P_t = 0, A_t = 2, \\ 1, & P_t = 0, A_t = 0, I_t = 0, \\ 1, & P_t = 0, A_t = 1, G_t = 0, \\ 0, & \text{otherwise.} \end{cases}$$

ICU utilisation is also recorded as a diagnostic signal:

$$U_{ICU}(S_t) = \begin{cases} 1, & I_t < I_{\max}, \\ 0, & I_t = I_{\max}. \end{cases}$$

In the small environment $I_{\max} = 2$, while in the capacity-increase experiments $I_{\max} = 10$. The same reward logic is preserved when the general-bed capacity increases from 3 to 30.

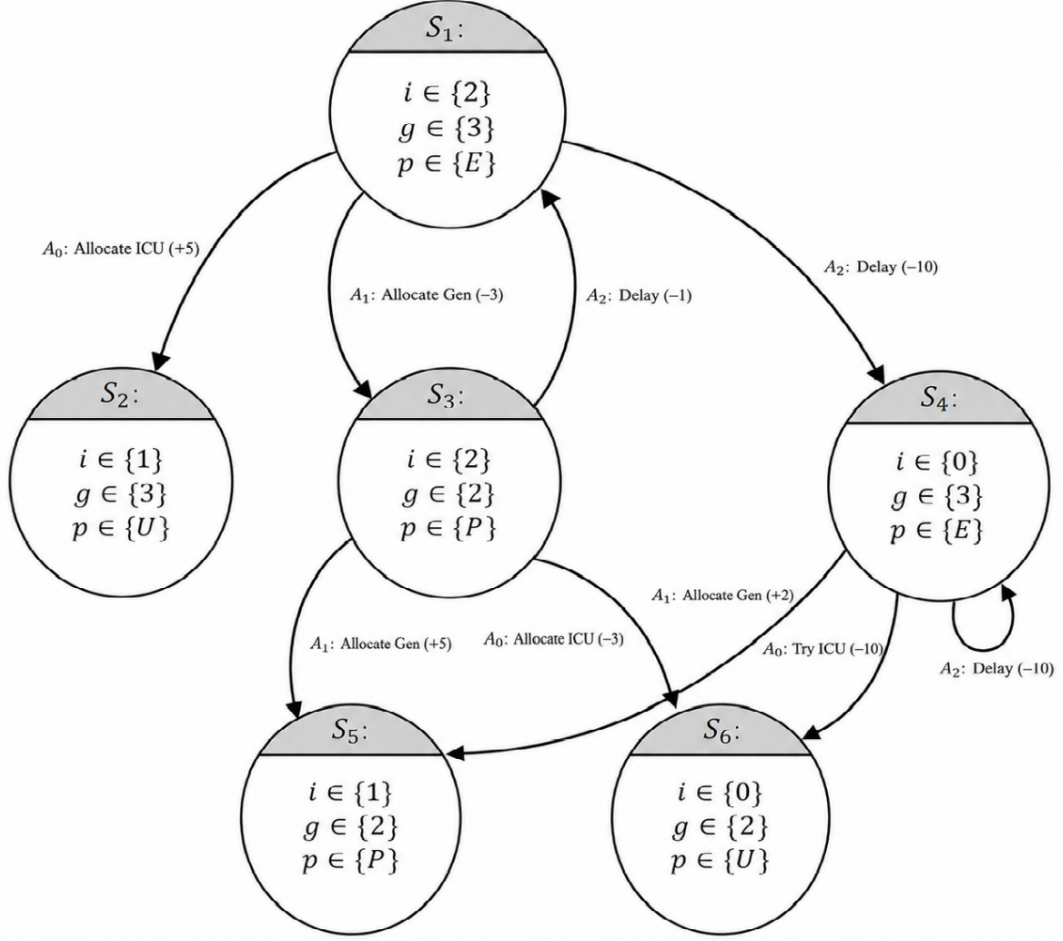


Figure B – State-transition diagram for the hospital bed-allocation MDP.

3. Q-Learning Setup

Q-table

The tabular Q-learning baseline uses a Q-table indexed by the discrete state and action. In the initial environment, the state size is $3 \times 4 \times 3 = 36$ states and the action size is 3. Therefore, the Q-table shape is 36×3 .

Parameters

- Discount factor $\gamma = 0.9$.
- Learning rate $\alpha = 0.1$.
- Exploration policy: ϵ -greedy.
- Start with high ϵ and decay over episodes so that the agent explores early and exploits learned actions later.

Q-learning update

For each transition, the Q-learning update moves the current Q-value towards the observed reward plus the discounted maximum Q-value of the next state. This allows the agent to improve its allocation policy over repeated simulated episodes.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

4. Q-Learning Performance

Running Q-learning and measuring performance

Episodes and horizon: an episode is a sequence of decisions over a “day” or “shift”, for example 50 steps. The original run uses $N = 1500$ episodes.

Per episode, the following metrics are tracked:

- Total reward.
- Number of admitted patients by type.
- Number of rejected or delayed emergency patients.
- Average waiting penalty.
- The results are then plotted using episode index against average return, usually smoothed for readability.

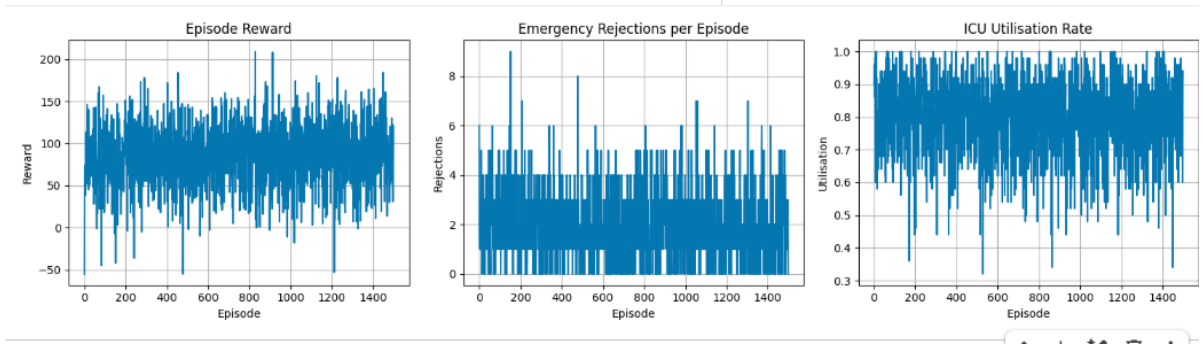


Figure C – Q-learning performance plots showing episode reward, emergency rejections and ICU utilisation.

Q-learning performance interpretation

The three graphs in the original draft give the episode reward, emergency rejections per episode and ICU utilisation. It can be seen on the episode reward that the performance is adequate but shows a lot of variation or noise. At some points the reward becomes negative, showing that the required positive measures of bed allocation are not being met. In a real situation this would mean that patient treatment is delayed.

This is further reinforced when looking at emergency rejections. There are significant peaks, which again have a negative impact on patient care because they mean patients are not admitted in a timely fashion.

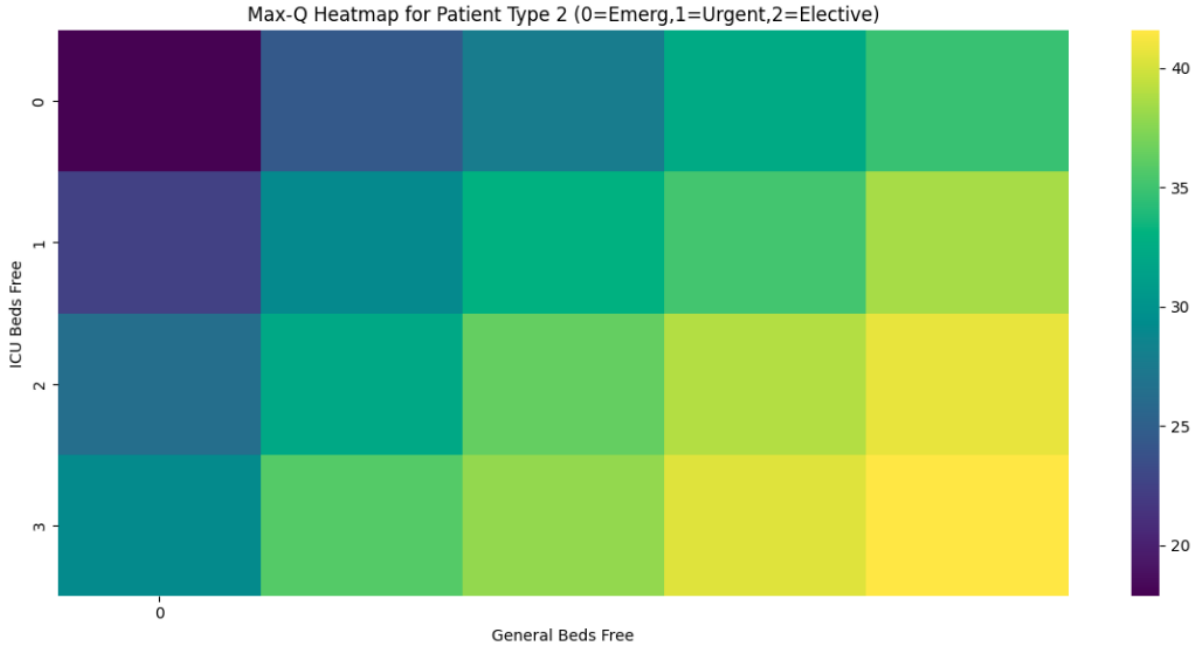


Figure – Q-table heatmap for ICU and general bed availability in the smaller environment.

The Q-table heatmap shows the reward for different states of free ICU and general beds. In the smaller environment, Q-learning favours the situation that gives the greatest reward, where the maximum number of ICU and general beds are free. This means that any new patient being admitted is more likely to have a bed available.

Overall, this is a very simple situation compared to the realities of a healthcare organisation. However, it shows the way that Q-learning can be used and also its limitations. As the number of beds of each type are increased, the number of states increases and tabular Q-learning begins to struggle.

For comparison, the original draft also refers to a heatmap for the elective patient type with ICU beds set to 10 and general beds set to 30.

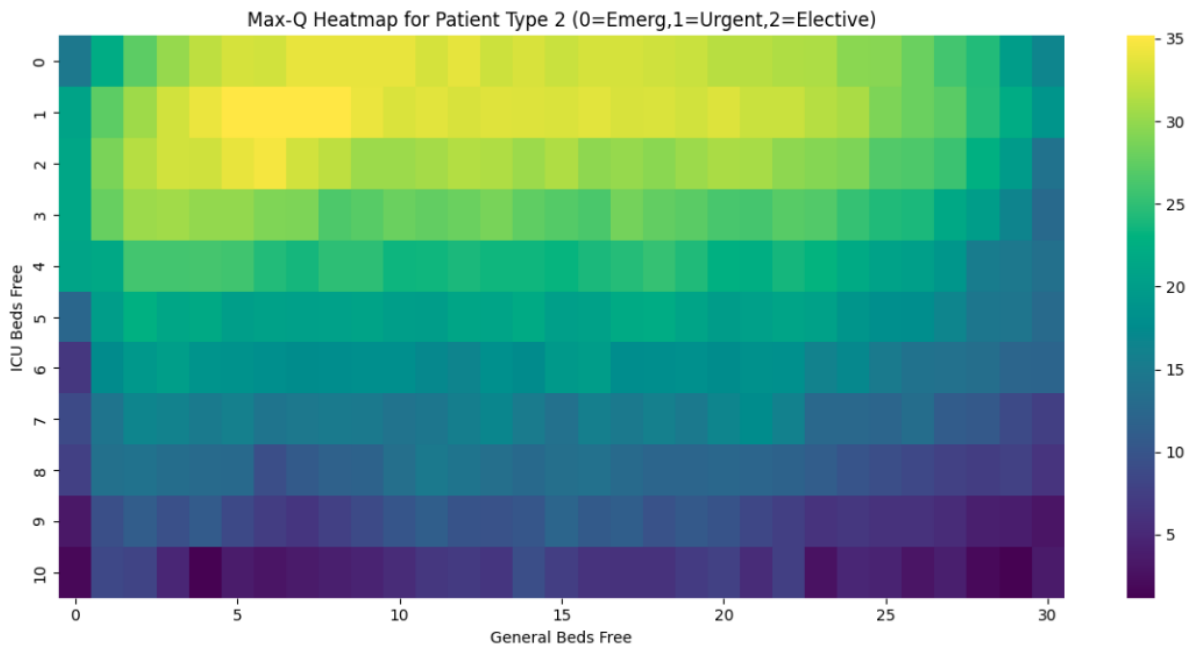


Figure – Q-table heatmap for the elective patient type with expanded ICU and general bed capacity.

In this situation, the heatmap favours both ICU and general capacity towards the lower end of the scale.

Parameter and Policy Experiments

So far the learning rate α has been fixed at 0.1 and the discount factor γ at 0.9. In order to determine the effect on the reward, these were varied. α was set at values of $\{0.1, 0.3, 0.5\}$ with a fixed value for $\gamma = 0.9$.

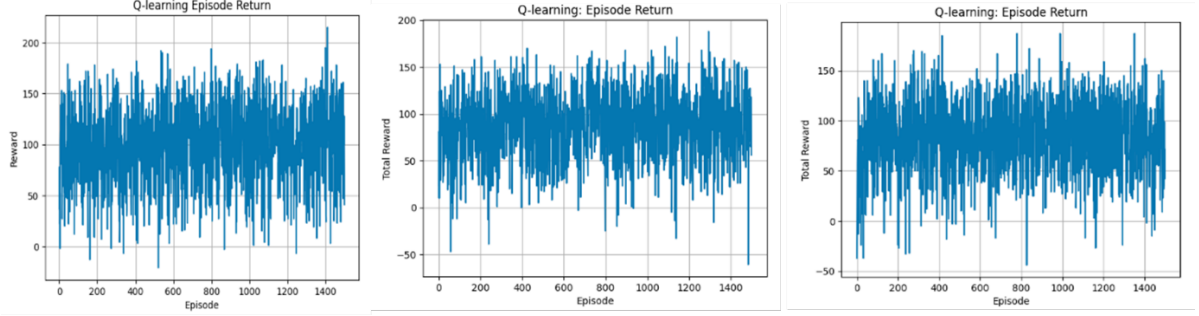


Figure – Q-learning learning-rate experiment with fixed discount factor.

The original value of $\alpha = 0.1$ gives the best overall results. The values of 0.3 and 0.5 show greater dips into negative reward, which would mean that patients are not admitted in a timely fashion. This is shown through the penalties in the reward signal.

Next the value of discount factor γ was varied between $\{0.5, 0.7, 0.9\}$, with $\alpha = 0.1$ fixed, which was shown to be the value that gave the most stable reward values.

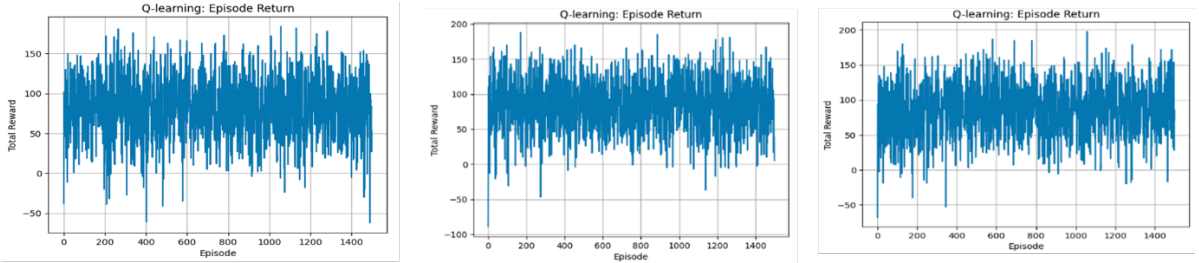


Figure – Q-learning discount-factor experiment with fixed learning rate.

From these results it can be seen that the discount value γ , which indicates how much the model cares about future rewards, has less of an effect on the reward value. The learning rate α , which shows how much new information overrides the old, has more of an effect. This shows that when the model learns more slowly it is more stable, which is expected from the lower value of α . The higher values of α show more instability.

These experiments support the use of $\alpha = 0.1$ and $\gamma = 0.9$ as the baseline configuration. However, final claims should be supported with tables showing mean reward, variance, emergency rejection count and ICU utilisation across multiple seeds.

Analysis of Q-Learning Results

The Q-learning baseline demonstrates that tabular reinforcement learning can capture basic decision logic in a small hospital bed-management environment. The agent learns that emergency patients

should be prioritised, that general beds should usually be used for elective patients, and that maintaining available capacity can improve future reward.

The main limitation is scalability. Tabular Q-learning stores a separate value estimate for every state-action pair. This is practical when the state space contains only 36 states, but it becomes increasingly inefficient as capacity levels, patient features, waiting times or ward types are added.

Another limitation is instability in performance. The reward curve and emergency-rejection metric show that the policy can still produce undesirable episodes. This is partly caused by stochastic patient arrivals and discharges, but it also reflects the limited generalisation ability of tabular Q-learning. Similar states are treated as separate entries rather than being learned through shared patterns.

Overall, Q-learning is a useful baseline because it is transparent and easy to interpret. However, its weaknesses justify moving to Deep Q-Networks, where a neural network can approximate the action-value function and generalise across larger or more continuous state representations.

Part B: Advanced - Deep Q-Network (Team)

7. DQN with Two Improvements

The environment for the first part of this testing was the same as for the Q-learning so that a comparison could be made. After that, the DQN environment was increased in complexity to show how DQN can scale to much larger problems compared to Q-learning.

The architecture for the DQN environment was chosen based on the parameters selected for the hospital environment.

The 5-dimensional input vector directly encodes the Markov state S_t , which is the number of ICU beds, the number of general beds and the 3 patient types of Emergencies, Urgent and Elective. The 3 outputs correspond to the discrete action set $A_t \in \{0, 1, 2\}$. The use of two hidden layers with 64 units was based on standard practice for this level of deep reinforcement learning problem. Using 32 units is generally shown to underfit, as opposed to 128 which tends to overfit. The two-layer MLP with ReLU is a universal approximator.

The initial tests started by directly comparing the results of the DQN and improvements with the tabular Q-learning method. Therefore, they employed a low-complexity model. As expected, the DQN showed significant improvement over tabular Q-learning in the original result graphs.

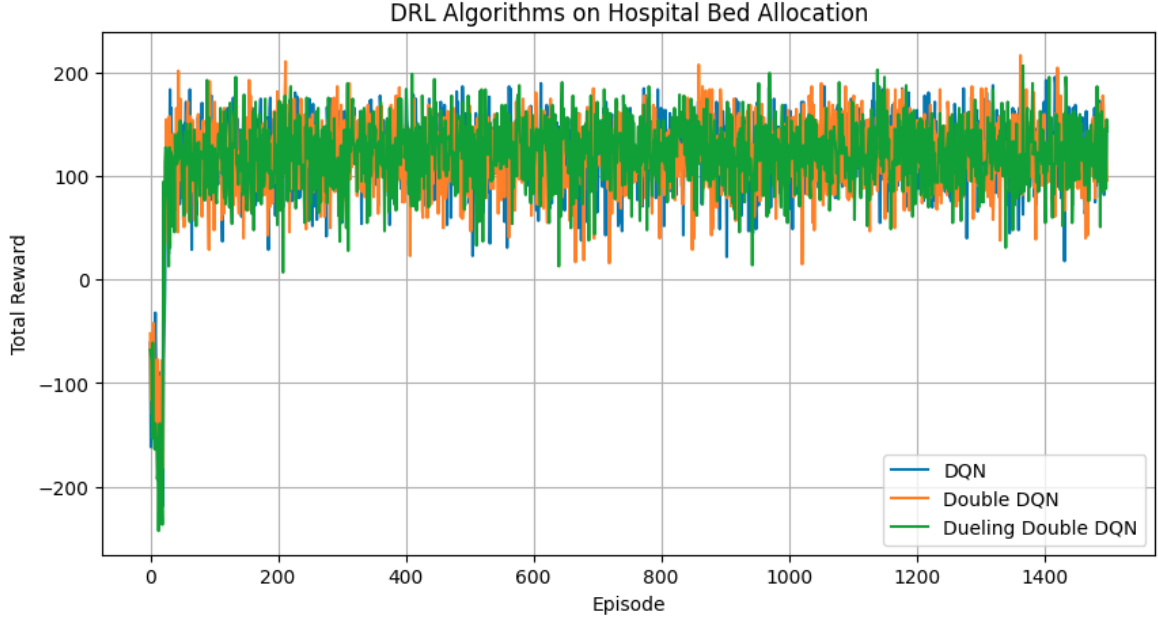


Figure H – Tabular Q-learning and DQN-family performance comparison.

The DQN agents show an increasing level of stability with less noise for each of the improvements. There is an early learning period when the networks are learning and show a lower level of performance compared to tabular Q. Once this period has passed, the network outperforms tabular Q.

Table 2: Base DQN architecture implemented for the hospital bed-allocation task.

Layer	Operation	Output size
Input	State vector ($I_t, G_t, \text{one-hot}(P_t)$)	5
Hidden 1	Linear(5, 64) + ReLU	64
Hidden 2	Linear(64, 64) + ReLU	64
Output	Linear(64, 3) Q-values	3

Table 3: DQN-family training hyperparameters used in the increased-capacity experiment.

Parameter	Implemented value
Training episodes	1,500
Episode length	50 steps
Discount factor γ	0.99
Learning rate	1×10^{-3}
Optimiser	Adam
Loss function	Mean squared error
Batch size	64
Replay-buffer capacity	10,000 transitions
Minimum buffer before updates	1,000 transitions
Initial exploration ϵ	1.0
Final exploration ϵ	0.05
Exploration decay	500
Target-network update frequency	50 episodes
ICU capacity	10 beds
General-bed capacity	30 beds

The base DQN, Double DQN and Dueling Double DQN experiments share the same state representation, action set and replay-buffer training structure. Double DQN keeps the same base network architecture but changes the target calculation by selecting the next action using the policy network and evaluating it using the target network. Dueling Double DQN changes the architecture by separating the value stream $V(s)$ and advantage stream $A(s, a)$, then combining them to estimate Q-values.

8. Analysis of DQN Results

To be able to clearly demonstrate the advantages of DQN techniques over tabular Q-learning, the environment complexity was increased as follows.

State space

Each state is represented as:

$$S_t = (I_t, G_t, P_t)$$

I_t = number of free ICU beds at time t

G_t = number of free general beds at time t

P_t = patient type at time t

Where:

- ICU free beds: 0 to 10.
- General free beds: 0 to 30.
- Patient type: 0 = Emergency, 1 = Urgent, 2 = Elective.

This represents a significant increase in the state space from 36 to $11 \times 31 \times 3 = 1023$.

Tabular Q-learning can still operate, but the state space is now large enough to justify deep RL.

For deep RL, states are encoded as a 5-dimensional vector. This continuous representation allows neural networks to generalise across capacity levels.

Action space

Three discrete actions are used:

- Allocate ICU bed.
- Allocate general bed.
- Delay admission.

Reward function

The reward structure encodes clinical and operational priorities:

Scenario	Reward
Allocate ICU to emergency/urgent	+5
Allocate ICU to elective	-3
Allocate general to elective	+5
Allocate general to emergency/urgent when ICU available	-3
Allocate general to emergency/urgent when ICU full	+2
Attempt allocation when full	-10
Delay any patient	-1
Delay emergency	-10 additional

Figure I – Reward structure for the increased-complexity DQN hospital bed-allocation environment.

This encourages ICU prioritisation for emergencies, general ward use for electives, avoiding unnecessary delays and avoiding misuse of ICU beds.

Transition dynamics

- Discharges: ICU +1 bed with probability 0.3.
- Discharges: General +1 bed with probability 0.4.
- New patient arrival distribution: $\Pr(P_{t+1} = \text{emergency}) = 0.2$, $\Pr(P_{t+1} = \text{urgent}) = 0.3$, $\Pr(P_{t+1} = \text{elective}) = 0.5$.
- Episodes last 50 steps, simulating a short operational window.

The result graphs in the original draft show the tabular Q-learning baseline with increased complexity and the DQN variants. All DQN methods show a stable, noise-reduced level of performance, with the Dueling Double DQN giving the best results.

The main interpretation is that DQN improves scalability and stability compared with tabular Q-learning, but it also introduces higher computational cost, more hyperparameters and less transparent decision logic. This trade-off should be discussed explicitly because hospital decision-support systems require both performance and interpretability.

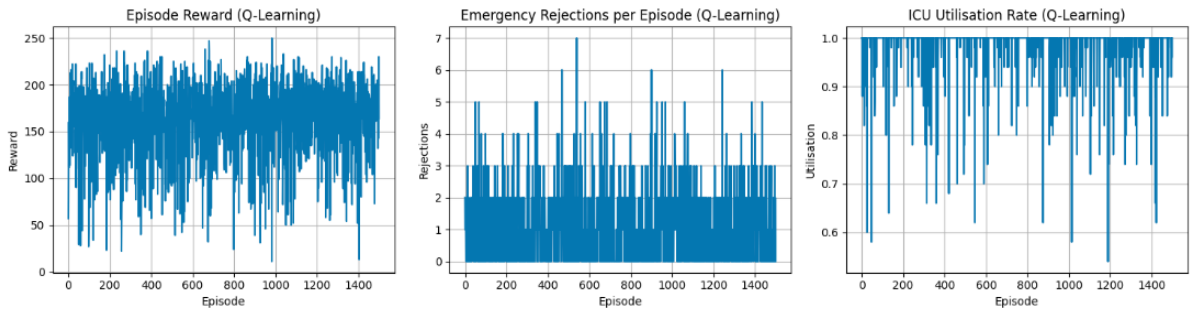


Figure – Q-learning performance plots for the increased-complexity environment with ICU = 10 and GEN = 30.

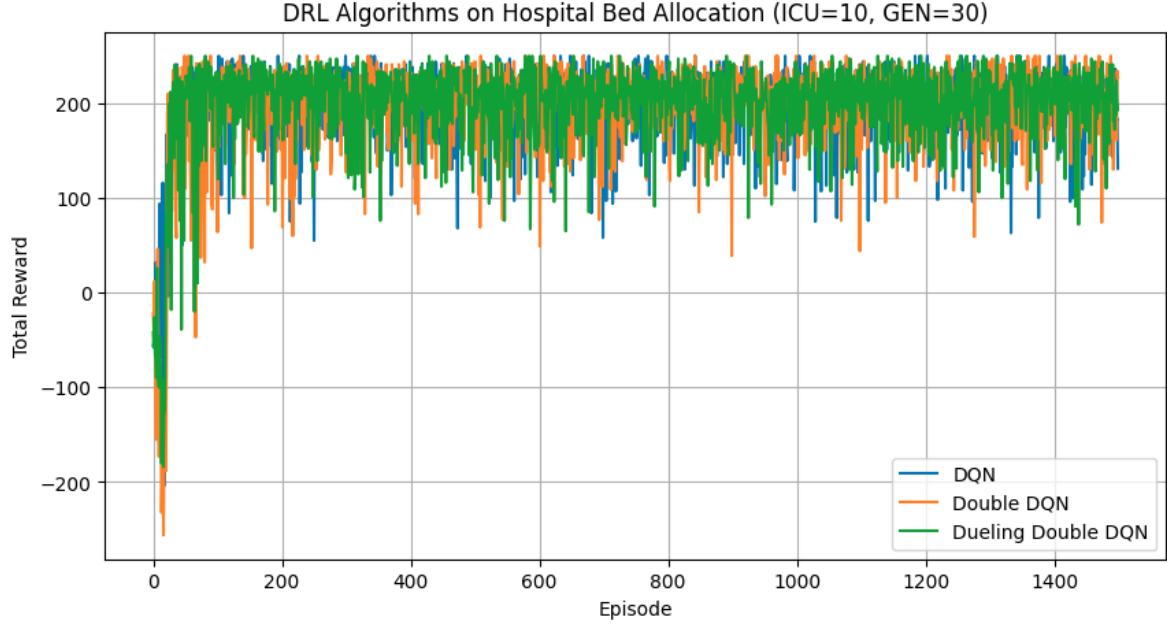


Figure – Reward comparison for DQN, Double DQN and Dueling Double DQN in the increased-capacity environment.

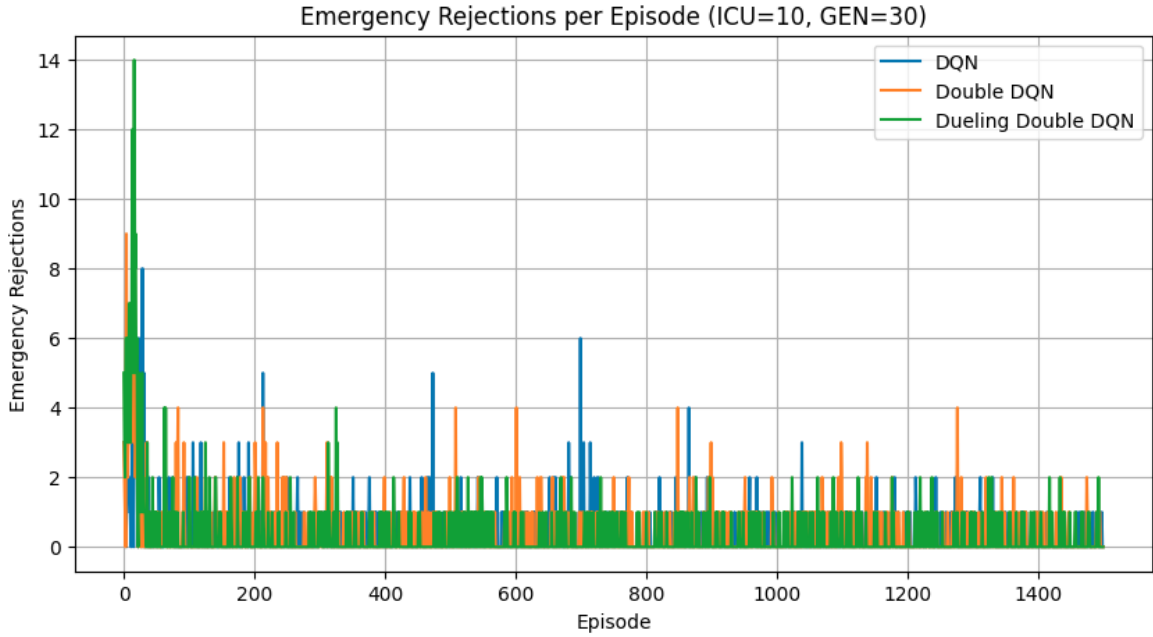


Figure – Emergency rejections per episode for DQN, Double DQN and Dueling Double DQN in the increased-capacity environment.

Part C: Individual - Advanced DRL Algorithm

9. Proximal Policy Optimization

As an extension to the deep RL comparison, Proximal Policy Optimization (PPO) was also employed in our experiments. It was selected specifically because PPO is a policy gradient-based algorithm. The other three DRL algorithms used, DQN, Double DQN and Dueling Double DQN are all value-based algorithms, where the network approximates values of taking actions in specific states, while

PPO directly attempts to approximate a policy for action selection. It does however approximate a value function to assist in updating its policy. PPO is therefore an appropriate choice of algorithm with which to compare the other three, as they all work in the same bed allocation environment. This bed allocation environment used was also unchanged, where the agent has access to the current hospital state, including the number of available beds and the type of patient. It then takes one of the three available actions, allocate an ICU bed, allocate a general ward bed or delay the patient. The reward function remains unchanged from that used previously to ensure that all differences in agent performance can be due to the algorithm, rather than the environment.

Due to our environment observations being numerical as opposed to images of a visual field of the environment, the PPO was implemented as a multilayer perceptron policy. We did not implement PPO using a convolutional network, since our environment is not a grid environment. PPO is trained using StableBaselines3, so we wrapped our environment with DummyVecEnv which provides the vectorized environment interface expected by Stable-Baselines3. Since our environment is so lightweight, this implementation was appropriate since it would be unlikely to result in any slowdown due to multiple processes.

The PPO algorithm itself works on an actor-critic basis, where the actor learns the policy for which actions are taken, and the critic learns to approximate the value of the current state for which an action has not been taken. PPO restricts how the policy is updated by using a clipped objective, which avoids very large policy updates that can lead to unstable behaviour. This is particularly helpful for this particular problem, as unstable updates could be undesirable, for instance, repeatedly delaying patients or allocating beds unnecessarily in the ICU. Once trained, the policy obtained for each configuration is then evaluated using 1500 episodes. All the models are trained for 30000 timesteps for each run. The PPO hyperparameters used are shown in Table 5.

Table 5: PPO hyperparameters.

Parameter	Value
Algorithm	PPO
Policy	MLP Policy
Total timesteps	30,000
Learning rate	0.0001
Gamma	0.99
Batch size	64
n_steps	256
Entropy coefficient	0.01

This evaluation phase consisted of 1500 episodes and monitored total episode reward, the count of emergency rejections, and the utilisation rate of ICU beds. These additional metrics were deemed necessary because mean reward alone does not accurately describe hospital performance, as a reward-maximising agent could be highly effective yet consistently exhibit hazardous actions like rejecting all patients deemed emergencies. Consequently, other relevant metrics needed to be considered to describe hospital performance, in conjunction with the performance of the reinforcement learning algorithm.

10. PPO Results and Analysis

Both quantitative and qualitative analysis was performed on the results generated by the PPO algorithm. Quantitative analysis was carried out based on the numerical metrics: the mean reward

and its corresponding standard deviation, the rejection rate of emergency patients and the ICU utilisation. Qualitative analysis was then conducted based on the patterns exhibited in the reward curve, diagnostic plots and policy heatmaps.

The overall PPO evaluation results are shown in Table 6.

Table 6: PPO evaluation results.

Metric	Value
Total timesteps	30,000
Evaluation episodes	1,500
Learning rate	0.0001
Batch size	64
Number of steps	256
Gamma	0.99
GAE lambda	0.95
Entropy coefficient	0.01
Mean reward	134.676
Reward standard deviation	29.96794525
Best reward	217
Worst reward	34
Emergency rejection	0.595333333
ICU utilisation	0.82772
Delayed patients	15.40466667
Random mean reward	-81.12333333
Random emergency rejection	4.746666667
Random ICU utilisation	0.670666667

Figure 1 displays the PPO episode reward curve, illustrating how the learned policy performs over a series of evaluation episodes. A high, stable curve suggests the agent has mastered an effective allocation strategy. Conversely, a noisy curve implies either high environment stochasticity or a lack of consistent policy generalisation.

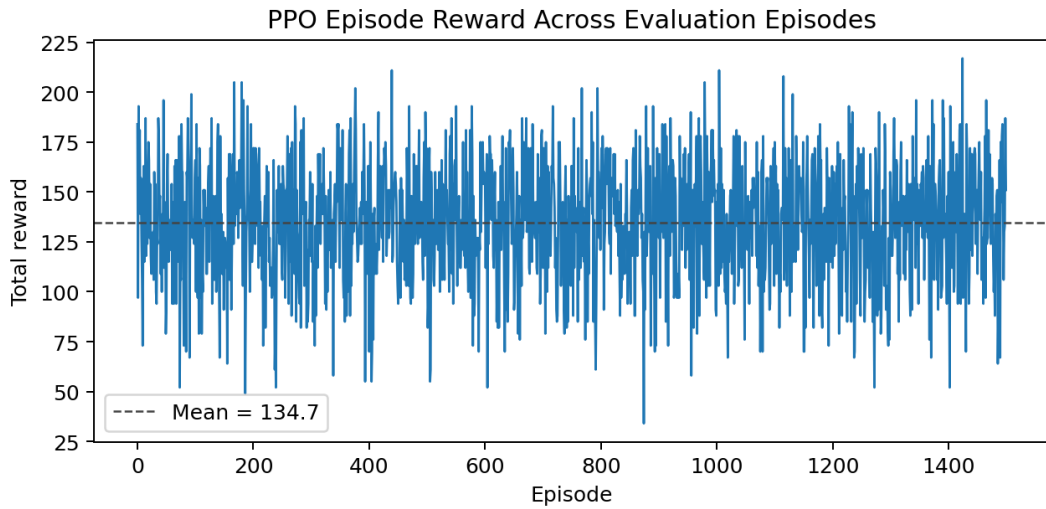


Figure 1: PPO episode return across evaluation episodes.

As seen in Figure F, PPO rewards across episodes ranged from 34 to 217. The average episode reward for the PPO agent was 134.676, indicating the learned policy developed a useful allocation strategy

when compared to the random baseline. However, despite the consistently positive reward curve, it remains noisy, suggesting performance varies across different realisations of stochastic patient arrivals.

Figure 2 resents the emergency rejection metric. This is a significant metric, as emergency rejection/delays entail a much higher cost than delaying other patients.

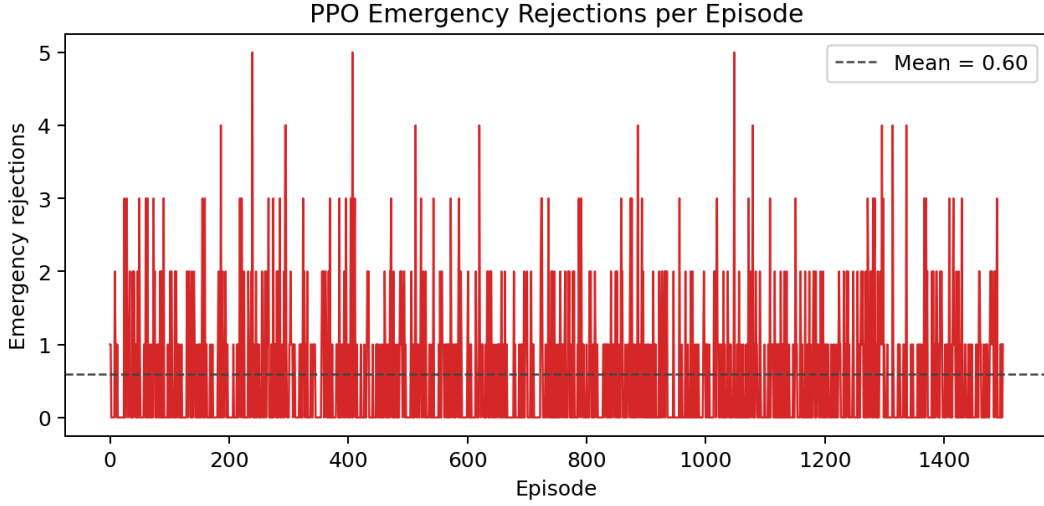


Figure 2: Emergency rejections per episode for PPO.

The average of 0.595 emergency rejections for the PPO agent shows the policy successfully reduced unsafe emergency handling compared to a random baseline with an average of 4.747 emergency rejections. Emergency rejection rate remains one of the primary metrics for evaluating policy performance.

Figure 3 presents the ICU utilisation metric.

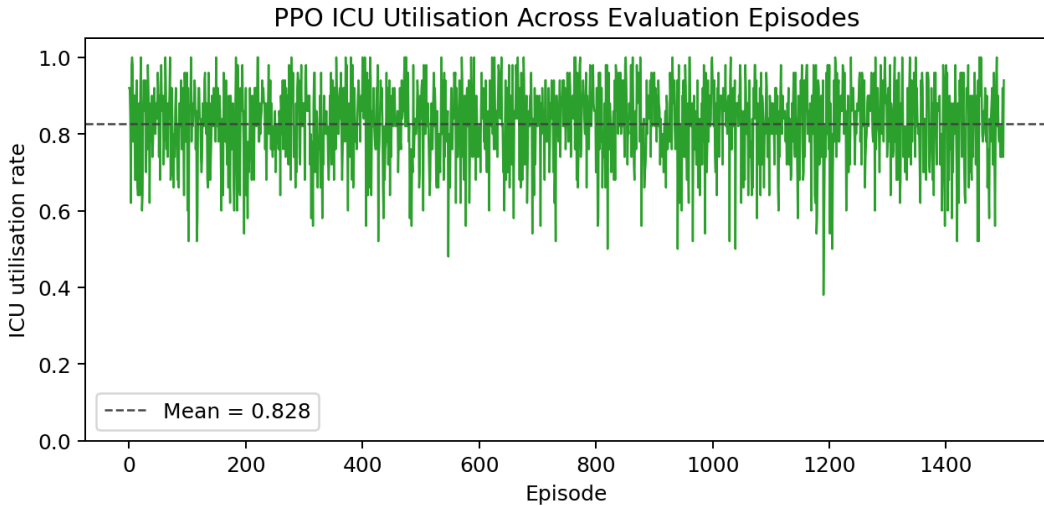


Figure 3: ICU utilisation rate for PPO.

The ICU utilisation was on average 0.828. Very low ICU utilisation indicates the agent is not using all available critical care resources; whereas, high ICU utilisation indicates the agent is overusing ICU beds for treatment, leaving very few ICU beds for future emergency patients. Thus, the optimal policy is not the one with the highest ICU utilisation rate. The best policy is likely the one

that optimises ICU utilisation while leaving a sufficient number of ICU beds available for future emergencies.

Comparing the final performance with the random policy baseline, the PPO agent performed better with a mean reward of 134.676 versus -81.123 for the baseline. This shows that, for this environment, the policy-gradient-based PPO agent is better at finding a useful policy than a random-policy baseline selecting actions at random. Further comparative analysis against the DQN-based agents would be conducted when their results are available.

Qualitatively, PPO appeared to learn a mixed allocation policy across patients rather than collapsing to one type of action. For the evaluated episodes, the policy allocated general patients most frequently (29808), followed by delay (23107), then ICU (22085). This distribution of action selection is appropriate as the mean reward is improved and the emergency rejection rate is lower than the random policy, though further analysis of delay actions is warranted as they are an important aspect of ICU operation.

Finally, PPO was evaluated by observing the policy’s preferred action in each patient arrival state via a heatmap. As Q-learning agents learn a table of Q-values for each state/action pair and PPO learns a policy, a Max-Q heatmap is not appropriate to evaluate the PPO agent. Instead, the following heatmap presents the action selected for each patient arrival state for the trained PPO policy.

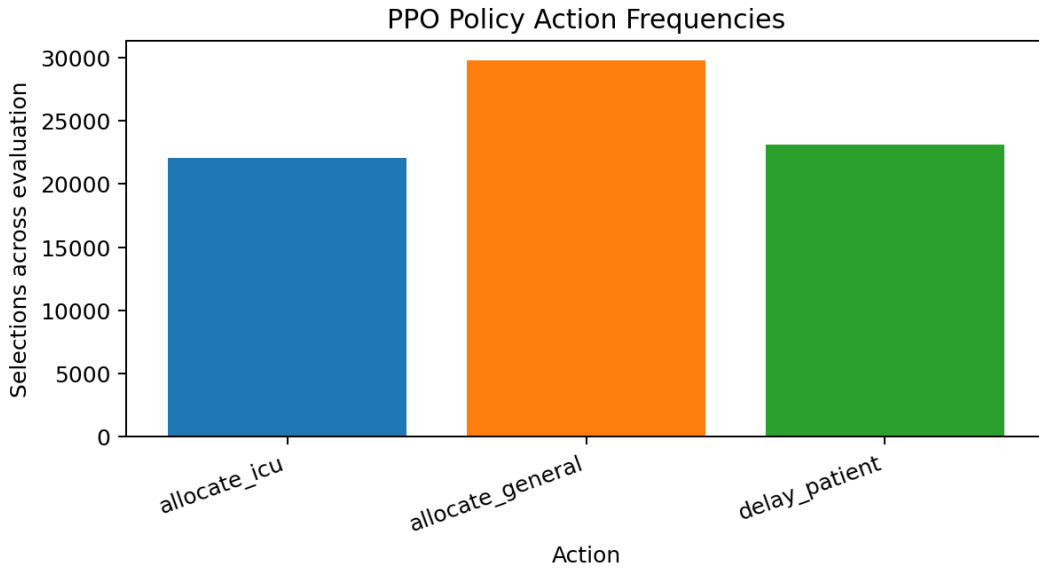


Figure 4: PPO policy-action diagnostic by patient type.

The available policy diagnostic indicates that PPO most frequently chooses the allocate_general option (29,808 times), while other actions are utilized during the evaluation process. Since a genuine state-sweep policy heatmap was not present in the saved files, we have exercised some caution in relying on this action-frequency snapshot of the results. In general, PPO represents an excellent comparison to the DQN-based methods in that it learns through direct policy optimization, as opposed to action-value estimation. The experimental results show that PPO was successful compared to the random baseline in the given environment. The greatest benefit was higher mean rewards and a reduction in emergency rejections; conversely, the biggest drawback was the persistently random actions observed during stochastic episode evaluation.

To conclude, from the first set of experiments it became increasingly evident that the design of the environment heavily influenced the behaviour of the policy; especially when it was over-complicated by irrelevant specifics which attempted to mimic a world model environment (and not to be confused

with a basic grid environment). As shown in Figure A, no heatmap could be generated as expected because the agent exhibited degenerate policy due to the absence of key actions that encourage exploration to reach the Reject or Admit states. Consequently, we reverted to using the original environment to produce quantitative results and usable data.

Finally, we used a series of different hyperparameter for training the agent, with grid search the chosen method for hyperparameter tuning as manual tuning was a lengthy exercise mainly for (Basic) and the first few runs of (Advanced) training. Specifically, we targeted the three most significant parameters, i.e., learning rate, batch size, and the number of training steps. We used the pre-determined values of 0.96 for lambda and 0.99 for Gamma, for the respective runs.

The results also indicate that algorithm choice alone does not ensure a safe allocation policy. Reward design, environment design and evaluation metrics all influence what the agent learns. Future experiments should consider more relevant hospital constraints, including length of stay, ward specialisation, cleaning delays, staff availability and multiple hospital units. This PPO experiment should therefore be considered a limited simulation rather than a clinically deployable decision-making system.

Contribution Summary

Task	Patrick Maw	Yehoshua Perez Condori
1. Environment Definition	Shared	Shared
2. Transition & Reward	Shared	Shared
3. Q-Learning Setup	Shared	Shared
4. Q-Learning Performance	Shared	Shared
5. Parameter Experiments	Shared	Shared
6. Q-Learning Analysis	Shared	Shared
7. DQN Implementation	Shared	Shared
8. DQN Analysis	Shared	Shared

GenAI Use Statement

Generative AI was used to support drafting, restructuring and proofreading of report text. The technical design, implementation, experimental outputs and final claims must be verified against the project code and saved result files. Any AI-assisted wording should be checked by the authors, and no result, figure, metric or implementation claim should be included unless it is supported by the repository outputs.

ChatGPT and Codex were used as writing and programming support tools during the preparation of this coursework. The assistance included restructuring draft report text, explaining reinforcement-learning code, improving notebook organisation, converting report content into LaTeX, checking that numerical claims matched saved notebook outputs, and helping to identify unsupported placeholders. The tools were not used as a source of experimental results: reported metrics, figures and algorithm claims were checked against the submitted notebooks, exported scripts and saved CSV/figure outputs. Final responsibility for the report content, interpretation and academic judgement remains with the authors.

References

- Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction. 2nd ed. MIT Press.
- Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. *Machine Learning*, 8, pp. 279-292.
- Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, pp. 529-533.
- van Hasselt, H., Guez, A. and Silver, D. (2016). Deep reinforcement learning with Double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Wang, Z. et al. (2016). Dueling network architectures for deep reinforcement learning. *Proceedings of the International Conference on Machine Learning*.
- Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
- NHS England Digital. Improving hospital bed allocation using AI. Available at: <https://digital.nhs.uk/services/ai-knowledge-repository/case-studies/improving-hospital-bed-allocation-using-ai>.
- GOV.UK. Using AI to improve patient bed allocation decisions in hospital. Available at: <https://www.gov.uk/government/case-studies/using-ai-to-improve-patient-bed-allocation-decisions-in-hospital>.
- NumPy Developers. NumPy documentation. Available at: <https://numpy.org/doc/>.
- pandas Developers. pandas documentation. Available at: <https://pandas.pydata.org/docs/>.
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.
- Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.
- Raffin, A. et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268), 1–8.
- Towers, M. et al. (2024). Gymnasium: A Standard Interface for Reinforcement Learning Environments. Available at: <https://gymnasium.farama.org/>.

Appendix

The following appendices include the Python scripts used for the coursework experiments. The files are included directly so that comments and indentation are preserved.

Basic Task Code: Steps 1., and 2. Rejection and Utilization Tracking

```
1  # %% [markdown]
2  # # DRL Coursework 1a: Rejection and Utilisation Tracking
3  # Extension that tracks emergency rejections and ICU utilisation alongside rewards.
4  #
5
6  # %% [markdown]
7  # ## Imports
8  # Libraries are grouped first so every later cell has the same dependencies available.
9  #
10
11 # %%
12 import random
13 from collections import deque
14 import torch
15 import numpy as np
16 import matplotlib.pyplot as plt
17 import torch.nn as nn
18 import torch.optim as optim
19
20
21 # %% [markdown]
22 # ## Reproducibility setup
23 # A fixed seed makes random, NumPy, and PyTorch sampling easier to reproduce.
24 #
25
26 # %%
27 SEED = 42
28
29 random.seed(SEED)
30 np.random.seed(SEED)
31 torch.manual_seed(SEED)
32 if torch.cuda.is_available():
33     torch.cuda.manual_seed_all(SEED)
34
35 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
36 print(f"Using device: {device}")
37
38
39 # %% [markdown]
40 # ## Environment definition
41 # The hospital environment stores bed availability, patient type, actions, rewards, and transitions.
42 #
43
44 # %%
45 class HospitalEnv:
46     def __init__(self):
47         # ICU free: 0,1,2
48         # GEN free: 0,1,2,3
49         # Patient type: 0=emergency, 1=urgent, 2=elective
50         self.reset()
51
52     def reset(self):
53         self.icu_free = np.random.randint(0, 3)
54         self.gen_free = np.random.randint(0, 4)
55         self.ptype = np.random.choice([0,1,2])
56         return self._get_state()
57
58     def _get_state(self):
```

```

59     return (self.icu_free, self.gen_free, self.ptype)
60
61     def step(self, action):
62         reward = 0
63         icu, gen, p = self.icu_free, self.gen_free, self.ptype
64
65         # APPLY ACTION
66         if action == 0: # allocate ICU
67             if icu > 0:
68                 self.icu_free -= 1
69                 if p in [0,1]: reward += 5
70                 else: reward -= 3
71             else:
72                 reward -= 10
73
74         elif action == 1: # allocate general
75             if gen > 0:
76                 self.gen_free -= 1
77                 if p == 2: reward += 5
78                 elif p in [0,1]:
79                     if icu > 0: reward -= 3
80                     else: reward += 2
81             else:
82                 reward -= 10
83
84         elif action == 2: # delay
85             reward -= 1
86             if p == 0:
87                 reward -= 10
88
89         # DISCHARGES
90         if random.random() < 0.3:
91             self.icu_free = min(self.icu_free + 1, 2)
92         if random.random() < 0.4:
93             self.gen_free = min(self.gen_free + 1, 3)
94
95         # EMERGENCY REJECTION LOGIC
96         emergency_rejected = 0
97         if p == 0: # emergency patient
98             if action == 0 and icu == 0:
99                 emergency_rejected = 1
100             elif action == 1 and gen == 0:
101                 emergency_rejected = 1
102             elif action == 2:
103                 emergency_rejected = 1
104
105         # ICU utilisation: 1 if at least one ICU bed is occupied
106         icu_occupied = 1 if self.icu_free < 2 else 0
107
108         # NEW PATIENT ARRIVAL
109         self.ptype = np.random.choice([0,1,2], p=[0.2,0.3,0.5])
110
111         next_state = self._get_state()
112         done = False
113         return next_state, reward, done, emergency_rejected, icu_occupied
114
115
116     # %% [markdown]
117     # ## Tabular Q-learning
118     # These helpers map states to Q-table rows and update values with epsilon-greedy exploration.
119     #
120
121     # %%
122     def state_to_index(s):
123         icu, gen, p = s
124         return icu*12 + gen*3 + p # 36 states
125
126
127     # %% [markdown]
128     # ## Tabular Q-learning

```

```

129 # These helpers map states to Q-table rows and update values with epsilon-greedy exploration.
130 #
131
132 # %%
133 def run_q_learning(episodes=1000, alpha=0.1, gamma=0.9, epsilon=0.2):
134     env = HospitalEnv()
135     Q = np.zeros((36, 3))
136
137     rewards_per_episode = []
138     emergency_rejections = []
139     icu_utilisation = []
140
141     for ep in range(episodes):
142         s = env.reset()
143         total_reward = 0
144         episode_rejections = 0
145         episode_icu_occ = 0
146
147         for t in range(50):
148             s_idx = state_to_index(s)
149
150             # epsilon-greedy
151             if random.random() < epsilon:
152                 a = np.random.randint(0, 3)
153             else:
154                 a = np.argmax(Q[s_idx])
155
156             s_next, r, done, rej, icu_occ = env.step(a)
157             s_next_idx = state_to_index(s_next)
158
159             # Q update
160             Q[s_idx, a] += alpha * (r + gamma * np.max(Q[s_next_idx]) - Q[s_idx, a])
161
162             s = s_next
163             total_reward += r
164             episode_rejections += rej
165             episode_icu_occ += icu_occ
166
167         rewards_per_episode.append(total_reward)
168         emergency_rejections.append(episode_rejections)
169         icu_utilisation.append(episode_icu_occ / 50.0)
170
171     return Q, rewards_per_episode, emergency_rejections, icu_utilisation
172
173
174 # %% [markdown]
175 # ## Q-learning experiment
176 # This cell runs the tabular baseline and plots reward, rejection, or utilisation traces.
177 #
178
179 # %%
180 # 3. RUN EXPERIMENT + PLOTS
181
182 Q, rewards, rejections, icu_util = run_q_learning(episodes=1500)
183
184 plt.figure(figsize=(15,4))
185
186 plt.subplot(1,3,1)
187 plt.plot(rewards)
188 plt.title("Episode Reward")
189 plt.xlabel("Episode")
190 plt.ylabel("Reward")
191 plt.grid(True)
192
193 plt.subplot(1,3,2)
194 plt.plot(rejections)
195 plt.title("Emergency Rejections per Episode")
196 plt.xlabel("Episode")
197 plt.ylabel("Rejections")
198 plt.grid(True)

```

```

199
200 plt.subplot(1,3,3)
201 plt.plot(icu_util)
202 plt.title("ICU Utilisation Rate")
203 plt.xlabel("Episode")
204 plt.ylabel("Utilisation")
205 plt.grid(True)
206
207 plt.tight_layout()
208 plt.show()
209
210
211
212 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
213
214
215 # %% [markdown]
216 # ## State vector representation
217 # Deep RL models use numeric state vectors instead of integer Q-table indices.
218 #
219
220 # %%
221 # State vector conversion
222 def state_to_vector(s):
223     icu, gen, p = s
224     p_onehot = np.zeros(3)
225     p_onehot[p] = 1.0
226     return np.array([icu, gen] + p_onehot.tolist(), dtype=np.float32)
227
228
229 # %% [markdown]
230 # ## Replay buffer
231 # Experience replay stores transitions so DQN updates can sample mixed past experience.
232 #
233
234 # %%
235 # Replay Buffer
236 class ReplayBuffer:
237     def __init__(self, capacity=10000):
238         self.buffer = deque(maxlen=capacity)
239
240     def push(self, state, action, reward, next_state, done):
241         self.buffer.append((state, action, reward, next_state, done))
242
243     def sample(self, batch_size):
244         batch = random.sample(self.buffer, batch_size)
245         states, actions, rewards, next_states, dones = zip(*batch)
246         return (
247             torch.tensor(states, dtype=torch.float32, device=device),
248             torch.tensor(actions, dtype=torch.long, device=device),
249             torch.tensor(rewards, dtype=torch.float32, device=device),
250             torch.tensor(next_states, dtype=torch.float32, device=device),
251             torch.tensor(dones, dtype=torch.float32, device=device),
252         )
253
254     def __len__(self):
255         return len(self.buffer)
256
257
258 # %% [markdown]
259 # ## DQN network models
260 # Standard and dueling networks estimate action values for each state.
261 #
262
263 # %%
264 # DQN Networks
265 class DQN(nn.Module):
266     def __init__(self, state_dim=5, action_dim=3, hidden_dim=64):
267         super(DQN, self).__init__()
268         self.net = nn.Sequential(

```

```

269         nn.Linear(state_dim, hidden_dim),
270         nn.ReLU(),
271         nn.Linear(hidden_dim, hidden_dim),
272         nn.ReLU(),
273         nn.Linear(hidden_dim, action_dim)
274     )
275
276     def forward(self, x):
277         return self.net(x)
278
279
280     # %% [markdown]
281     # ## DQN network models
282     # Standard and dueling networks estimate action values for each state.
283     #
284
285     # %%
286     class DuelingDQN(nn.Module):
287         def __init__(self, state_dim=5, action_dim=3, hidden_dim=64):
288             super(DuelingDQN, self).__init__()
289             self.feature = nn.Sequential(
290                 nn.Linear(state_dim, hidden_dim),
291                 nn.ReLU()
292             )
293             self.value_stream = nn.Sequential(
294                 nn.Linear(hidden_dim, hidden_dim),
295                 nn.ReLU(),
296                 nn.Linear(hidden_dim, 1)
297             )
298             self.adv_stream = nn.Sequential(
299                 nn.Linear(hidden_dim, hidden_dim),
300                 nn.ReLU(),
301                 nn.Linear(hidden_dim, action_dim)
302             )
303
304             def forward(self, x):
305                 feat = self.feature(x)
306                 value = self.value_stream(feat)
307                 adv = self.adv_stream(feat)
308                 q = value + adv - adv.mean(dim=1, keepdim=True)
309                 return q
310
311
312     # %% [markdown]
313     # ## DQN training loop
314     # The training loop supports standard DQN, Double DQN, and Dueling Double DQN.
315     #
316
317     # %%
318     # TRAINING FUNCTION WITH NEW METRICS
319     def train_dqn(
320         episodes=1000,
321         gamma=0.99,
322         lr=1e-3,
323         batch_size=64,
324         buffer_capacity=10000,
325         min_buffer_size=1000,
326         epsilon_start=1.0,
327         epsilon_end=0.05,
328         epsilon_decay=500,
329         target_update_freq=50,
330         double_dqn=True,
331         dueling=False,
332         icu_capacity=2
333     ):
334         env = HospitalEnv()
335         state_dim = 5
336         action_dim = 3
337
338         if dueling:

```

```

339     policy_net = DuelingDQN(state_dim, action_dim).to(device)
340     target_net = DuelingDQN(state_dim, action_dim).to(device)
341 else:
342     policy_net = DQN(state_dim, action_dim).to(device)
343     target_net = DQN(state_dim, action_dim).to(device)
344
345 target_net.load_state_dict(policy_net.state_dict())
346 target_net.eval()
347
348 optimizer = optim.Adam(policy_net.parameters(), lr=lr)
349 replay_buffer = ReplayBuffer(capacity=buffer_capacity)
350
351 rewards_per_episode = []
352 emergency_rej_per_episode = []
353 icu_util_per_episode = []
354
355 steps_done = 0
356
357 for ep in range(epochs):
358     s = env.reset()
359     s_vec = state_to_vector(s)
360     total_reward = 0
361
362     emergency_rej = 0
363     icu_util_sum = 0.0
364     step_count = 0
365
366     for t in range(50):
367         epsilon = epsilon_end + (epsilon_start - epsilon_end) * np.exp(-steps_done / epsilon_decay)
368         steps_done += 1
369
370         if random.random() < epsilon:
371             a = np.random.randint(0, action_dim)
372         else:
373             with torch.no_grad():
374                 state_t = torch.tensor(np.array([s_vec]), device=device) # Fixed: Convert to numpy
array first
375                 q_values = policy_net(state_t)
376                 a = q_values.argmax(dim=1).item()
377
378                 # ICU utilisation: calculated based on the state *before* the action
379                 icu_free, gen_free, ptype = s
380                 icu_occupied_beds = icu_capacity - icu_free
381                 icu_util_sum += icu_occupied_beds / icu_capacity
382                 step_count += 1
383
384                 # Call env.step and unpack all 5 return values
385                 # The 5 values are: next_state, reward, done, emergency_rejected, icu_occupied_flag
386                 # We use 'emergency_rejected_from_env' for the metric and discard 'icu_occupied_flag_from_env'
387                 s_next, r, done, emergency_rejected_from_env, _ = env.step(a)
388
389                 # Update emergency rejection metric using the value from env.step
390                 emergency_rej += emergency_rejected_from_env
391
392                 s_next_vec = state_to_vector(s_next)
393
394                 replay_buffer.push(s_vec, a, r, s_next_vec, done)
395
396                 s_vec = s_next_vec
397                 s = s_next
398                 total_reward += r
399
400     if len(replay_buffer) >= min_buffer_size:
401         states, actions, rewards, next_states, dones = replay_buffer.sample(batch_size)
402
403         q_values = policy_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)
404
405         with torch.no_grad():
406             if double_dqn:
407                 next_q_policy = policy_net(next_states)

```

```

408         next_actions = next_q_policy.argmax(dim=1, keepdim=True)
409         next_q_target = target_net(next_states).gather(1, next_actions).squeeze(1)
410     else:
411         next_q_target = target_net(next_states).max(dim=1)[0]
412
413     targets = rewards + gamma * (1 - dones) * next_q_target
414
415     loss = nn.MSELoss()(q_values, targets)
416     optimizer.zero_grad()
417     loss.backward()
418     optimizer.step()
419
420     if done:
421         break
422
423     rewards_per_episode.append(total_reward)
424     emergency_rej_per_episode.append(emergency_rej)
425     icu_util_per_episode.append(icu_util_sum / step_count)
426
427     if ep % target_update_freq == 0:
428         target_net.load_state_dict(policy_net.state_dict())
429
430     return rewards_per_episode, emergency_rej_per_episode, icu_util_per_episode
431
432
433 # %% [markdown]
434 # ## Deep RL experiment runs
435 # This cell trains the configured deep RL variants for comparison.
436 #
437
438 # %%
439 # RUN EXPERIMENTS
440 rewards_dqn, emrej_dqn, icu_dqn = train_dqn(
441     episodes=1500,
442     double_dqn=False,
443     dueling=False
444 )
445
446 rewards_double, emrej_double, icu_double = train_dqn(
447     episodes=1500,
448     double_dqn=True,
449     dueling=False
450 )
451
452 rewards_dueling, emrej_dueling, icu_dueling = train_dqn(
453     episodes=1500,
454     double_dqn=True,
455     dueling=True
456 )
457
458
459 # %% [markdown]
460 # ## Result plots
461 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
462 #
463
464 # %%
465 # PLOTS
466
467 # Reward plot
468 plt.figure(figsize=(10,5))
469 plt.plot(rewards_dqn, label="DQN")
470 plt.plot(rewards_double, label="Double DQN")
471 plt.plot(rewards_dueling, label="Dueling Double DQN")
472 plt.xlabel("Episode")
473 plt.ylabel("Total Reward")
474 plt.title("DRL Algorithms on Hospital Bed Allocation")
475 plt.legend()
476 plt.grid(True)
477 plt.show()

```

```

478
479
480 # %% [markdown]
481 # ## Result plots
482 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
483 #
484
485 # %%
486 # Emergency rejections
487 plt.figure(figsize=(10,5))
488 plt.plot(emrej_dqn, label="DQN")
489 plt.plot(emrej_double, label="Double DQN")
490 plt.plot(emrej_dueling, label="Dueling Double DQN")
491 plt.xlabel("Episode")
492 plt.ylabel("Emergency Rejections")
493 plt.title("Emergency Rejections per Episode")
494 plt.legend()
495 plt.grid(True)
496 plt.show()
497
498
499 # %% [markdown]
500 # ## Result plots
501 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
502 #
503
504 # %%
505 # ICU utilisation
506 plt.figure(figsize=(10,5))
507 plt.plot(icu_dqn, label="DQN")
508 plt.plot(icu_double, label="Double DQN")
509 plt.plot(icu_dueling, label="Dueling Double DQN")
510 plt.xlabel("Episode")
511 plt.ylabel("ICU Utilisation")
512 plt.title("ICU Utilisation per Episode")
513 plt.legend()
514 plt.grid(True)
515 plt.show()

```

Listing 1: Basic Task Code: Rejection and Utilisation Tracking

Basic Task: Steps 3., and 4. - DQN and Double DQN

```

1  # %% [markdown]
2  # # DRL Coursework 1a: Capacity-Increase Variant
3  # Large-capacity version that discretises state values before learning.
4  #
5
6  # %% [markdown]
7  # ## Imports
8  # Libraries are grouped first so every later cell has the same dependencies available.
9  #
10
11 # %%
12 import random
13 from collections import deque
14 import torch
15 import numpy as np
16 import matplotlib.pyplot as plt
17 import torch.nn as nn
18 import torch.optim as optim
19
20
21 # %% [markdown]
22 # ## Reproducibility setup

```

```

23 # A fixed seed makes random, NumPy, and PyTorch sampling easier to reproduce.
24 #
25
26 # %%
27 SEED = 42
28
29 random.seed(SEED)
30 np.random.seed(SEED)
31 torch.manual_seed(SEED)
32 if torch.cuda.is_available():
33     torch.cuda.manual_seed_all(SEED)
34
35 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
36 print(f"Using device: {device}")
37
38
39 # %% [markdown]
40 # ## Environment definition
41 # The hospital environment stores bed availability, patient type, actions, rewards, and transitions.
42 #
43
44 # %%
45 class HospitalEnv:
46     def __init__(self, icu_capacity=10, gen_capacity=30):
47         # ICU free: 0..icu_capacity
48         # GEN free: 0..gen_capacity
49         # Patient type: 0=emergency, 1=urgent, 2=elective
50         self.icu_capacity = icu_capacity
51         self.gen_capacity = gen_capacity
52         self.reset()
53
54     def reset(self):
55         self.icu_free = np.random.randint(0, self.icu_capacity + 1)
56         self.gen_free = np.random.randint(0, self.gen_capacity + 1)
57         self.ptype = np.random.choice([0, 1, 2])
58         return self._get_state()
59
60     def _get_state(self):
61         return (self.icu_free, self.gen_free, self.ptype)
62
63     def step(self, action):
64         reward = 0
65         icu, gen, p = self.icu_free, self.gen_free, self.ptype
66
67         # APPLY ACTION
68         if action == 0: # allocate ICU
69             if icu > 0:
70                 self.icu_free -= 1
71                 if p in [0, 1]:
72                     reward += 5
73                 else:
74                     reward -= 3
75             else:
76                 reward -= 10
77
78         elif action == 1: # allocate general
79             if gen > 0:
80                 self.gen_free -= 1
81                 if p == 2:
82                     reward += 5
83                 elif p in [0, 1]:
84                     if icu > 0:
85                         reward -= 3
86                     else:
87                         reward += 2
88             else:
89                 reward -= 10
90
91         elif action == 2: # delay
92             reward -= 1

```

```

93         if p == 0:
94             reward -= 10
95
96         # DISCHARGES
97         if random.random() < 0.3:
98             self.icu_free = min(self.icu_free + 1, self.icu_capacity)
99         if random.random() < 0.4:
100             self.gen_free = min(self.gen_free + 1, self.gen_capacity)
101
102         # EMERGENCY REJECTION LOGIC
103         emergency_rejected = 0
104         if p == 0: # emergency patient
105             if action == 0 and icu == 0:
106                 emergency_rejected = 1
107             elif action == 1 and gen == 0:
108                 emergency_rejected = 1
109             elif action == 2:
110                 emergency_rejected = 1
111
112         # ICU utilisation: 1 if at least one ICU bed is occupied
113         icu_occupied = 1 if self.icu_free < self.icu_capacity else 0
114
115         # NEW PATIENT ARRIVAL
116         self.ptype = np.random.choice([0, 1, 2], p=[0.2, 0.3, 0.5])
117
118         next_state = self._get_state()
119         done = False
120         return next_state, reward, done, emergency_rejected, icu_occupied
121
122
123     # %% [markdown]
124     # ## Tabular Q-learning
125     # These helpers map states to Q-table rows and update values with epsilon-greedy exploration.
126     #
127
128     # %%
129     def discretise_state(s, icu_capacity=10, gen_capacity=30,
130                         icu_bins=(0, 1, 3, 6, 10),
131                         gen_bins=(0, 5, 10, 20, 30)):
132         icu_free, gen_free, p = s
133
134         # map icu_free into bin index
135         icu_bin = 0
136         for i, b in enumerate(icu_bins):
137             if icu_free <= b:
138                 icu_bin = i
139                 break
140         else:
141             icu_bin = len(icu_bins) - 1
142
143         # map gen_free into bin index
144         gen_bin = 0
145         for j, b in enumerate(gen_bins):
146             if gen_free <= b:
147                 gen_bin = j
148                 break
149         else:
150             gen_bin = len(gen_bins) - 1
151
152         return icu_bin, gen_bin, p
153
154
155     # %% [markdown]
156     # ## Tabular Q-learning
157     # These helpers map states to Q-table rows and update values with epsilon-greedy exploration.
158     #
159
160     # %%
161     def state_to_index(s,
162                       icu_bins=(0, 1, 3, 6, 10),

```

```

163         gen_bins=(0, 5, 10, 20, 30)):
164     icu_bin, gen_bin, p = discretise_state(s, icu_bins=icu_bins, gen_bins=gen_bins)
165     n_icu = len(icu_bins)
166     n_gen = len(gen_bins)
167     # index = icu_bin * (n_gen * 3) + gen_bin * 3 + p
168     return icu_bin * (n_gen * 3) + gen_bin * 3 + p
169
170
171 # %% [markdown]
172 # ## Tabular Q-learning
173 # These helpers map states to Q-table rows and update values with epsilon-greedy exploration.
174 #
175
176 # %%
177 def run_q_learning(episodes=1000, alpha=0.1, gamma=0.9, epsilon=0.2):
178     env = HospitalEnv(icu_capacity=10, gen_capacity=30)
179
180     icu_bins = (0, 1, 3, 6, 10)
181     gen_bins = (0, 5, 10, 20, 30)
182     n_icu = len(icu_bins)
183     n_gen = len(gen_bins)
184     n_states = n_icu * n_gen * 3 # 3 patient types
185     n_actions = 3
186
187     Q = np.zeros((n_states, n_actions))
188
189     rewards_per_episode = []
190     emergency_rejections = []
191     icu_utilisation = []
192
193     for ep in range(episodes):
194         s = env.reset()
195         total_reward = 0
196         episode_rejections = 0
197         episode_icu_occ = 0
198
199         for t in range(50):
200             s_idx = state_to_index(s, icu_bins=icu_bins, gen_bins=gen_bins)
201
202             # epsilon-greedy
203             if random.random() < epsilon:
204                 a = np.random.randint(0, n_actions)
205             else:
206                 a = np.argmax(Q[s_idx])
207
208             s_next, r, done, rej, icu_occ = env.step(a)
209             s_next_idx = state_to_index(s_next, icu_bins=icu_bins, gen_bins=gen_bins)
210
211             # Q update
212             Q[s_idx, a] += alpha * (r + gamma * np.max(Q[s_next_idx]) - Q[s_idx, a])
213
214             s = s_next
215             total_reward += r
216             episode_rejections += rej
217             episode_icu_occ += icu_occ
218
219             rewards_per_episode.append(total_reward)
220             emergency_rejections.append(episode_rejections)
221             icu_utilisation.append(episode_icu_occ / 50.0)
222
223     return Q, rewards_per_episode, emergency_rejections, icu_utilisation
224
225
226 # %% [markdown]
227 # ## Q-learning experiment
228 # This cell runs the tabular baseline and plots reward, rejection, or utilisation traces.
229 #
230
231 # %%
232 # 3. RUN Q-LEARNING EXPERIMENT + PLOTS

```

```

233
234 Q, rewards, rejections, icu_util = run_q_learning(episodes=1500)
235
236 plt.figure(figsize=(15, 4))
237
238 plt.subplot(1, 3, 1)
239 plt.plot(rewards)
240 plt.title("Episode Reward (Q-Learning)")
241 plt.xlabel("Episode")
242 plt.ylabel("Reward")
243 plt.grid(True)
244
245 plt.subplot(1, 3, 2)
246 plt.plot(rejections)
247 plt.title("Emergency Rejections per Episode (Q-Learning)")
248 plt.xlabel("Episode")
249 plt.ylabel("Rejections")
250 plt.grid(True)
251
252 plt.subplot(1, 3, 3)
253 plt.plot(icu_util)
254 plt.title("ICU Utilisation Rate (Q-Learning)")
255 plt.xlabel("Episode")
256 plt.ylabel("Utilisation")
257 plt.grid(True)
258
259 plt.tight_layout()
260 plt.show()
261
262
263 # %% [markdown]
264 # ## Supporting code
265 # This code supports the surrounding experiment section.
266 #
267
268 # %%
269 # 4. DEEP RL (DQN / DOUBLE DQN / DUELING DQN)
270
271
272
273 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
274
275
276 # %% [markdown]
277 # ## State vector representation
278 # Deep RL models use numeric state vectors instead of integer Q-table indices.
279 #
280
281 # %%
282 # State vector conversion
283 def state_to_vector(s, icu_capacity=10, gen_capacity=30):
284     icu, gen, p = s
285     # Normalise free beds by capacity
286     icu_norm = icu / icu_capacity
287     gen_norm = gen / gen_capacity
288     p_onehot = np.zeros(3, dtype=np.float32)
289     p_onehot[p] = 1.0
290     return np.array([icu_norm, gen_norm] + p_onehot.tolist(), dtype=np.float32)
291
292
293 # %% [markdown]
294 # ## Replay buffer
295 # Experience replay stores transitions so DQN updates can sample mixed past experience.
296 #
297
298 # %%
299 # Replay Buffer
300 class ReplayBuffer:
301     def __init__(self, capacity=10000):
302         self.buffer = deque(maxlen=capacity)

```

```

303
304     def push(self, state, action, reward, next_state, done):
305         self.buffer.append((state, action, reward, next_state, done))
306
307     def sample(self, batch_size):
308         batch = random.sample(self.buffer, batch_size)
309         states, actions, rewards, next_states, dones = zip(*batch)
310         return (
311             torch.tensor(states, dtype=torch.float32, device=device),
312             torch.tensor(actions, dtype=torch.long, device=device),
313             torch.tensor(rewards, dtype=torch.float32, device=device),
314             torch.tensor(next_states, dtype=torch.float32, device=device),
315             torch.tensor(dones, dtype=torch.float32, device=device),
316         )
317
318     def __len__(self):
319         return len(self.buffer)
320
321
322     ## [markdown]
323     ## DQN network models
324     Standard and dueling networks estimate action values for each state.
325     #
326
327     ##
328     DQN Networks
329     class DQN(nn.Module):
330         def __init__(self, state_dim=5, action_dim=3, hidden_dim=64):
331             super(DQN, self).__init__()
332             self.net = nn.Sequential(
333                 nn.Linear(state_dim, hidden_dim),
334                 nn.ReLU(),
335                 nn.Linear(hidden_dim, hidden_dim),
336                 nn.ReLU(),
337                 nn.Linear(hidden_dim, action_dim)
338             )
339
340         def forward(self, x):
341             return self.net(x)
342
343
344     ## [markdown]
345     ## DQN network models
346     Standard and dueling networks estimate action values for each state.
347     #
348
349     ##
350     class DuelingDQN(nn.Module):
351         def __init__(self, state_dim=5, action_dim=3, hidden_dim=64):
352             super(DuelingDQN, self).__init__()
353             self.feature = nn.Sequential(
354                 nn.Linear(state_dim, hidden_dim),
355                 nn.ReLU()
356             )
357             self.value_stream = nn.Sequential(
358                 nn.Linear(hidden_dim, hidden_dim),
359                 nn.ReLU(),
360                 nn.Linear(hidden_dim, 1)
361             )
362             self.adv_stream = nn.Sequential(
363                 nn.Linear(hidden_dim, hidden_dim),
364                 nn.ReLU(),
365                 nn.Linear(hidden_dim, action_dim)
366             )
367
368         def forward(self, x):
369             feat = self.feature(x)
370             value = self.value_stream(feat)
371             adv = self.adv_stream(feat)
372             q = value + adv - adv.mean(dim=1, keepdim=True)

```

```

373         return q
374
375
376     # %% [markdown]
377     # ## DQN training loop
378     # The training loop supports standard DQN, Double DQN, and Dueling Double DQN.
379     #
380
381     # %%
382     # TRAINING FUNCTION WITH NEW METRICS
383     def train_dqn(
384         episodes=1000,
385         gamma=0.99,
386         lr=1e-3,
387         batch_size=64,
388         buffer_capacity=10000,
389         min_buffer_size=1000,
390         epsilon_start=1.0,
391         epsilon_end=0.05,
392         epsilon_decay=500,
393         target_update_freq=50,
394         double_dqn=True,
395         dueling=False,
396         icu_capacity=10,
397         gen_capacity=30
398     ):
399         env = HospitalEnv(icu_capacity=icu_capacity, gen_capacity=gen_capacity)
400         state_dim = 5
401         action_dim = 3
402
403         if dueling:
404             policy_net = DuelingDQN(state_dim, action_dim).to(device)
405             target_net = DuelingDQN(state_dim, action_dim).to(device)
406         else:
407             policy_net = DQN(state_dim, action_dim).to(device)
408             target_net = DQN(state_dim, action_dim).to(device)
409
410         target_net.load_state_dict(policy_net.state_dict())
411         target_net.eval()
412
413         optimizer = optim.Adam(policy_net.parameters(), lr=lr)
414         replay_buffer = ReplayBuffer(capacity=buffer_capacity)
415
416         rewards_per_episode = []
417         emergency_rej_per_episode = []
418         icu_util_per_episode = []
419
420         steps_done = 0
421
422         for ep in range(episodes):
423             s = env.reset()
424             s_vec = state_to_vector(s, icu_capacity=icu_capacity, gen_capacity=gen_capacity)
425             total_reward = 0
426
427             emergency_rej = 0
428             icu_util_sum = 0.0
429             step_count = 0
430
431             for t in range(50):
432                 epsilon = epsilon_end + (epsilon_start - epsilon_end) * np.exp(-steps_done / epsilon_decay)
433                 steps_done += 1
434
435                 if random.random() < epsilon:
436                     a = np.random.randint(0, action_dim)
437                 else:
438                     with torch.no_grad():
439                         state_t = torch.tensor(np.array([s_vec]), device=device)
440                         q_values = policy_net(state_t)
441                         a = q_values.argmax(dim=1).item()
442

```

```

443     # ICU utilisation: calculated based on the state *before* the action
444     icu_free, gen_free, ptype = s
445     icu_occupied_beds = icu_capacity - icu_free
446     icu_util_sum += icu_occupied_beds / icu_capacity
447     step_count += 1
448
449     # env.step
450     s_next, r, done, emergency_rejected_from_env, _ = env.step(a)
451     emergency_rej += emergency_rejected_from_env
452
453     s_next_vec = state_to_vector(s_next, icu_capacity=icu_capacity, gen_capacity=gen_capacity)
454
455     replay_buffer.push(s_vec, a, r, s_next_vec, done)
456
457     s_vec = s_next_vec
458     s = s_next
459     total_reward += r
460
461     if len(replay_buffer) >= min_buffer_size:
462         states, actions, rewards, next_states, dones = replay_buffer.sample(batch_size)
463
464         q_values = policy_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)
465
466         with torch.no_grad():
467             if double_dqn:
468                 next_q_policy = policy_net(next_states)
469                 next_actions = next_q_policy.argmax(dim=1, keepdim=True)
470                 next_q_target = target_net(next_states).gather(1, next_actions).squeeze(1)
471             else:
472                 next_q_target = target_net(next_states).max(dim=1)[0]
473
474             targets = rewards + gamma * (1 - dones) * next_q_target
475
476             loss = nn.MSELoss()(q_values, targets)
477             optimizer.zero_grad()
478             loss.backward()
479             optimizer.step()
480
481         if done:
482             break
483
484         rewards_per_episode.append(total_reward)
485         emergency_rej_per_episode.append(emergency_rej)
486         icu_util_per_episode.append(icu_util_sum / step_count)
487
488         if ep % target_update_freq == 0:
489             target_net.load_state_dict(policy_net.state_dict())
490
491     return rewards_per_episode, emergency_rej_per_episode, icu_util_per_episode
492
493
494 # %% [markdown]
495 # ## Deep RL experiment runs
496 # This cell trains the configured deep RL variants for comparison.
497 #
498
499 # %%
500 # 5. RUN DEEP RL EXPERIMENTS
501 rewards_dqn, emrej_dqn, icu_dqn = train_dqn(
502     episodes=1500,
503     double_dqn=False,
504     dueling=False,
505     icu_capacity=10,
506     gen_capacity=30
507 )
508
509 rewards_double, emrej_double, icu_double = train_dqn(
510     episodes=1500,
511     double_dqn=True,
512     dueling=False,

```

```

513     icu_capacity=10,
514     gen_capacity=30
515 )
516
517 rewards_dueling, emrej_dueling, icu_dueling = train_dqn(
518     episodes=1500,
519     double_dqn=True,
520     dueling=True,
521     icu_capacity=10,
522     gen_capacity=30
523 )
524
525
526 # %% [markdown]
527 # ## Result plots
528 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
529 #
530
531 # %%
532 # 6. PLOTS
533
534 # Reward plot
535 plt.figure(figsize=(10, 5))
536 plt.plot(rewards_dqn, label="DQN")
537 plt.plot(rewards_double, label="Double DQN")
538 plt.plot(rewards_dueling, label="Dueling Double DQN")
539 plt.xlabel("Episode")
540 plt.ylabel("Total Reward")
541 plt.title("DRL Algorithms on Hospital Bed Allocation (ICU=10, GEN=30)")
542 plt.legend()
543 plt.grid(True)
544 plt.show()
545
546
547 # %% [markdown]
548 # ## Result plots
549 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
550 #
551
552 # %%
553 # Emergency rejections
554 plt.figure(figsize=(10, 5))
555 plt.plot(emrej_dqn, label="DQN")
556 plt.plot(emrej_double, label="Double DQN")
557 plt.plot(emrej_dueling, label="Dueling Double DQN")
558 plt.xlabel("Episode")
559 plt.ylabel("Emergency Rejections")
560 plt.title("Emergency Rejections per Episode (ICU=10, GEN=30)")
561 plt.legend()
562 plt.grid(True)
563 plt.show()
564
565
566 # %% [markdown]
567 # ## Result plots
568 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
569 #
570
571 # %%
572 # ICU utilisation
573 plt.figure(figsize=(10, 5))
574 plt.plot(icu_dqn, label="DQN")
575 plt.plot(icu_double, label="Double DQN")
576 plt.plot(icu_dueling, label="Dueling Double DQN")
577 plt.xlabel("Episode")
578 plt.ylabel("ICU Utilisation")
579 plt.title("ICU Utilisation per Episode (ICU=10, GEN=30)")
580 plt.legend()
581 plt.grid(True)
582 plt.show()

```

Listing 2: Capacity-Increase Code

Advanced Task: Steps 8., and 9. - Actor-Critic Algorithm Patrick Contribution

```
1  # %% [markdown]
2  # # DRL Coursework 1b: Combined Actor-Critic Variant
3  # Alternate Actor-Critic version using a combined actor-critic network.
4  #
5
6  # %% [markdown]
7  # ## Imports
8  # Libraries are grouped first so every later cell has the same dependencies available.
9  #
10
11 # %%
12 import random
13 import torch
14 from collections import deque
15 import numpy as np
16 import matplotlib.pyplot as plt
17 import torch.nn as nn
18 import torch.optim as optim
19
20
21 # %% [markdown]
22 # ## Reproducibility setup
23 # A fixed seed makes random, NumPy, and PyTorch sampling easier to reproduce.
24 #
25
26 # %%
27 SEED = 42
28
29 random.seed(SEED)
30 np.random.seed(SEED)
31 torch.manual_seed(SEED)
32 if torch.cuda.is_available():
33     torch.cuda.manual_seed_all(SEED)
34
35 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
36 print(f"Using device: {device}")
37
38
39 # %% [markdown]
40 # ## Supporting code
41 # This code supports the surrounding experiment section.
42 #
43
44 # %%
45 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
46
47
48 # %% [markdown]
49 # ## Environment definition
50 # The hospital environment stores bed availability, patient type, actions, rewards, and transitions.
51 #
52
53 # %%
54 class HospitalEnv:
55     def __init__(self):
56         # ICU free: 0,1,2
57         # GEN free: 0,1,2,3
58         # Patient type: 0=emergency, 1=urgent, 2=elective
59         self.reset()
60
```

```

61     def reset(self):
62         self.icu_free = np.random.randint(0, 3)
63         self.gen_free = np.random.randint(0, 4)
64         self.ptype = np.random.choice([0,1,2])
65         return self._get_state()
66
67     def _get_state(self):
68         return (self.icu_free, self.gen_free, self.ptype)
69
70     def step(self, action):
71         reward = 0
72         icu, gen, p = self.icu_free, self.gen_free, self.ptype
73
74         # APPLY ACTION
75         if action == 0: # allocate ICU
76             if icu > 0:
77                 self.icu_free -= 1
78                 if p in [0,1]: reward += 5
79                 else: reward -= 3
80             else:
81                 reward -= 10
82
83         elif action == 1: # allocate general
84             if gen > 0:
85                 self.gen_free -= 1
86                 if p == 2: reward += 5
87                 elif p in [0,1]:
88                     if icu > 0: reward -= 3
89                     else: reward += 2
90             else:
91                 reward -= 10
92
93         elif action == 2: # delay
94             reward -= 1
95             if p == 0:
96                 reward -= 10
97
98         # DISCHARGES
99         if random.random() < 0.3:
100             self.icu_free = min(self.icu_free + 1, 2)
101         if random.random() < 0.4:
102             self.gen_free = min(self.gen_free + 1, 3)
103
104         # EMERGENCY REJECTION LOGIC
105         emergency_rejected = 0
106         if p == 0: # emergency patient
107             if action == 0 and icu == 0:
108                 emergency_rejected = 1
109             elif action == 1 and gen == 0:
110                 emergency_rejected = 1
111             elif action == 2:
112                 emergency_rejected = 1
113
114         # ICU utilisation: 1 if at least one ICU bed is occupied
115         icu_occupied = 1 if self.icu_free < 2 else 0
116
117         # NEW PATIENT ARRIVAL
118         self.ptype = np.random.choice([0,1,2], p=[0.2,0.3,0.5])
119
120         next_state = self._get_state()
121         done = False
122         return next_state, reward, done, emergency_rejected, icu_occupied
123
124
125     # %% [markdown]
126     # ## Tabular Q-learning
127     # These helpers map states to Q-table rows and update values with epsilon-greedy exploration.
128     #
129
130     # %%

```

```

131 def state_to_index(s):
132     icu, gen, p = s
133     return icu*12 + gen*3 + p # 36 states
134
135
136 # %% [markdown]
137 # ## Tabular Q-learning
138 # These helpers map states to Q-table rows and update values with epsilon-greedy exploration.
139 #
140
141 # %%
142 def run_q_learning(episodes=1000, alpha=0.1, gamma=0.9, epsilon=0.2):
143     env = HospitalEnv()
144     Q = np.zeros((36, 3))
145
146     rewards_per_episode = []
147     emergency_rejections = []
148     icu_utilisation = []
149
150     for ep in range(episodes):
151         s = env.reset()
152         total_reward = 0
153         episode_rejections = 0
154         episode_icu_occ = 0
155
156         for t in range(50):
157             s_idx = state_to_index(s)
158
159             # epsilon-greedy
160             if random.random() < epsilon:
161                 a = np.random.randint(0, 3)
162             else:
163                 a = np.argmax(Q[s_idx])
164
165             s_next, r, done, rej, icu_occ = env.step(a)
166             s_next_idx = state_to_index(s_next)
167
168             # Q update
169             Q[s_idx, a] += alpha * (r + gamma * np.max(Q[s_next_idx]) - Q[s_idx, a])
170
171             s = s_next
172             total_reward += r
173             episode_rejections += rej
174             episode_icu_occ += icu_occ
175
176             rewards_per_episode.append(total_reward)
177             emergency_rejections.append(episode_rejections)
178             icu_utilisation.append(episode_icu_occ / 50.0)
179
180         return Q, rewards_per_episode, emergency_rejections, icu_utilisation
181
182
183 # %% [markdown]
184 # ## State vector representation
185 # Deep RL models use numeric state vectors instead of integer Q-table indices.
186 #
187
188 # %%
189 def state_to_vector(s):
190     icu, gen, p = s
191     p_onehot = np.zeros(3)
192     p_onehot[p] = 1.0
193     return np.array([icu, gen] + p_onehot.tolist(), dtype=np.float32)
194
195
196 # %% [markdown]
197 # ## Replay buffer
198 # Experience replay stores transitions so DQN updates can sample mixed past experience.
199 #
200

```

```

201 # %%
202 class ReplayBuffer:
203     def __init__(self, capacity=10000):
204         self.buffer = deque(maxlen=capacity)
205
206     def push(self, state, action, reward, next_state, done):
207         self.buffer.append((state, action, reward, next_state, done))
208
209     def sample(self, batch_size):
210         batch = random.sample(self.buffer, batch_size)
211         states, actions, rewards, next_states, dones = zip(*batch)
212         return (
213             torch.tensor(states, dtype=torch.float32, device=device),
214             torch.tensor(actions, dtype=torch.long, device=device),
215             torch.tensor(rewards, dtype=torch.float32, device=device),
216             torch.tensor(next_states, dtype=torch.float32, device=device),
217             torch.tensor(dones, dtype=torch.float32, device=device),
218         )
219
220     def __len__(self):
221         return len(self.buffer)
222
223
224 # %% [markdown]
225 # ## DQN network models
226 # Standard and dueling networks estimate action values for each state.
227 #
228
229 # %%
230 class DQN(nn.Module):
231     def __init__(self, state_dim=5, action_dim=3, hidden_dim=64):
232         super(DQN, self).__init__()
233         self.net = nn.Sequential(
234             nn.Linear(state_dim, hidden_dim),
235             nn.ReLU(),
236             nn.Linear(hidden_dim, hidden_dim),
237             nn.ReLU(),
238             nn.Linear(hidden_dim, action_dim)
239         )
240
241     def forward(self, x):
242         return self.net(x)
243
244
245 # %% [markdown]
246 # ## DQN network models
247 # Standard and dueling networks estimate action values for each state.
248 #
249
250 # %%
251 class DuelingDQN(nn.Module):
252     def __init__(self, state_dim=5, action_dim=3, hidden_dim=64):
253         super(DuelingDQN, self).__init__()
254         self.feature = nn.Sequential(
255             nn.Linear(state_dim, hidden_dim),
256             nn.ReLU()
257         )
258         self.value_stream = nn.Sequential(
259             nn.Linear(hidden_dim, hidden_dim),
260             nn.ReLU(),
261             nn.Linear(hidden_dim, 1)
262         )
263         self.adv_stream = nn.Sequential(
264             nn.Linear(hidden_dim, hidden_dim),
265             nn.ReLU(),
266             nn.Linear(hidden_dim, action_dim)
267         )
268
269     def forward(self, x):
270         feat = self.feature(x)

```

```

271         value = self.value_stream(feat)
272         adv = self.adv_stream(feat)
273         q = value + adv - adv.mean(dim=1, keepdim=True)
274         return q
275
276
277     # %% [markdown]
278     # ## DQN training loop
279     # The training loop supports standard DQN, Double DQN, and Dueling Double DQN.
280     #
281
282     # %%
283     def train_dqn(
284         episodes=1000,
285         gamma=0.99,
286         lr=1e-3,
287         batch_size=64,
288         buffer_capacity=10000,
289         min_buffer_size=1000,
290         epsilon_start=1.0,
291         epsilon_end=0.05,
292         epsilon_decay=500,
293         target_update_freq=50,
294         double_dqn=True,
295         dueling=False
296     ):
297         env = HospitalEnv()
298         state_dim = 5
299         action_dim = 3
300
301         if dueling:
302             policy_net = DuelingDQN(state_dim, action_dim).to(device)
303             target_net = DuelingDQN(state_dim, action_dim).to(device)
304         else:
305             policy_net = DQN(state_dim, action_dim).to(device)
306             target_net = DQN(state_dim, action_dim).to(device)
307
308         target_net.load_state_dict(policy_net.state_dict())
309         target_net.eval()
310
311         optimizer = optim.Adam(policy_net.parameters(), lr=lr)
312         replay_buffer = ReplayBuffer(capacity=buffer_capacity)
313
314         rewards_per_episode = []
315         steps_done = 0
316
317         for ep in range(episodes):
318             s = env.reset()
319             s_vec = state_to_vector(s)
320             total_reward = 0
321
322             for t in range(50):
323                 epsilon = epsilon_end + (epsilon_start - epsilon_end) * \
324                     np.exp(-1.0 * steps_done / epsilon_decay)
325                 steps_done += 1
326
327                 if random.random() < epsilon:
328                     a = np.random.randint(0, action_dim)
329                 else:
330                     with torch.no_grad():
331                         state_t = torch.tensor([s_vec], device=device)
332                         q_values = policy_net(state_t)
333                         a = q_values.argmax(dim=1).item()
334
335                 s_next, r, done, _, _ = env.step(a)
336                 s_next_vec = state_to_vector(s_next)
337
338                 replay_buffer.push(s_vec, a, r, s_next_vec, done)
339
340                 s_vec = s_next_vec

```

```

341         total_reward += r
342
343         if len(replay_buffer) >= min_buffer_size:
344             states, actions, rewards, next_states, dones = replay_buffer.sample(batch_size)
345
346             q_values = policy_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)
347
348             with torch.no_grad():
349                 if double_dqn:
350                     next_q_policy = policy_net(next_states)
351                     next_actions = next_q_policy.argmax(dim=1, keepdim=True)
352                     next_q_target = target_net(next_states).gather(1, next_actions).squeeze(1)
353                 else:
354                     next_q_target = target_net(next_states).max(dim=1)[0]
355
356             targets = rewards + gamma * (1 - dones) * next_q_target
357
358             loss = nn.MSELoss()(q_values, targets)
359
360             optimizer.zero_grad()
361             loss.backward()
362             optimizer.step()
363
364             if done:
365                 break
366
367             rewards_per_episode.append(total_reward)
368
369             if ep % target_update_freq == 0:
370                 target_net.load_state_dict(policy_net.state_dict())
371
372         return rewards_per_episode
373
374
375     # %% [markdown]
376     # ## Actor-Critic networks
377     # Actor-Critic models learn a policy and a value estimate instead of only Q-values.
378     #
379
380     # %%
381     class ActorCritic(nn.Module):
382         def __init__(self, state_dim=5, action_dim=3, hidden_dim=64):
383             super(ActorCritic, self).__init__()
384             self.shared = nn.Sequential(
385                 nn.Linear(state_dim, hidden_dim),
386                 nn.ReLU()
387             )
388             self.actor = nn.Sequential(
389                 nn.Linear(hidden_dim, hidden_dim),
390                 nn.ReLU(),
391                 nn.Linear(hidden_dim, action_dim)
392             )
393             self.critic = nn.Sequential(
394                 nn.Linear(hidden_dim, hidden_dim),
395                 nn.ReLU(),
396                 nn.Linear(hidden_dim, 1)
397             )
398
399         def forward(self, x):
400             feat = self.shared(x)
401             logits = self.actor(feat)
402             value = self.critic(feat)
403             return logits, value
404
405
406     # %% [markdown]
407     # ## Actor-Critic training loop
408     # This loop updates policy and value losses from simulated hospital episodes.
409     #
410

```

```

411 # %%
412 def train_actor_critic(
413     episodes=1000,
414     gamma=0.99,
415     lr=1e-3,
416     entropy_coef=0.01
417 ):
418     env = HospitalEnv()
419     state_dim = 5
420     action_dim = 3
421
422     model = ActorCritic(state_dim, action_dim).to(device)
423     optimizer = optim.Adam(model.parameters(), lr=lr)
424
425     rewards_per_episode = []
426     rejections_per_episode = []
427     icu_utilisation_per_episode = []
428
429     for ep in range(episodes):
430         s = env.reset()
431         s_vec = state_to_vector(s)
432
433         total_reward = 0
434         total_rejections = 0
435         total_icu_occ = 0
436
437         log_probs = []
438         values = []
439         rewards = []
440         entropies = []
441
442         for t in range(50):
443             state_t = torch.tensor([s_vec], dtype=torch.float32, device=device)
444             logits, value = model(state_t)
445
446             dist = torch.distributions.Categorical(logits=logits)
447             action = dist.sample()
448
449             log_prob = dist.log_prob(action)
450             entropy = dist.entropy()
451
452             s_next, r, done, rej, icu_occ = env.step(action.item())
453             s_next_vec = state_to_vector(s_next)
454
455             log_probs.append(log_prob)
456             values.append(value.squeeze(0))
457             rewards.append(torch.tensor(r, dtype=torch.float32, device=device))
458             entropies.append(entropy)
459
460             total_reward += r
461             total_rejections += rej
462             total_icu_occ += icu_occ
463
464             s_vec = s_next_vec
465
466             if done:
467                 break
468
469         returns = []
470         G = torch.tensor(0.0, device=device)
471         for r in reversed(rewards):
472             G = r + gamma * G
473             returns.insert(0, G)
474
475         returns = torch.stack(returns)
476         values = torch.stack(values)
477         log_probs = torch.stack(log_probs)
478         entropies = torch.stack(entropies)
479
480         advantages = returns - values.squeeze(-1)

```

```

481         actor_loss = -(log_probs * advantages.detach()).mean()
482         critic_loss = advantages.pow(2).mean()
483         entropy_loss = -entropies.mean()
484
485     loss = actor_loss + 0.5 * critic_loss + entropy_coef * entropy_loss
486
487     optimizer.zero_grad()
488     loss.backward()
489     optimizer.step()
490
491     rewards_per_episode.append(total_reward)
492     rejections_per_episode.append(total_rejections)
493     icu_utilisation_per_episode.append(total_icu_occ / 50.0)
494
495     return rewards_per_episode, rejections_per_episode, icu_utilisation_per_episode
496
497
498 # %% [markdown]
499 # ## Q-learning experiment
500 # This cell runs the tabular baseline and plots reward, rejection, or utilisation traces.
501 #
502
503 # %%
504 # 5. RUN EXPERIMENTS
505
506 # Q-learning
507 Q, rewards_q, rejections_q, icu_q = run_q_learning(episodes=1500)
508
509 plt.figure(figsize=(15,4))
510 plt.subplot(1,3,1)
511 plt.plot(rewards_q)
512 plt.title("Q-Learning: Episode Reward")
513 plt.xlabel("Episode")
514 plt.ylabel("Reward")
515 plt.grid(True)
516
517 plt.subplot(1,3,2)
518 plt.plot(rejections_q)
519 plt.title("Q-Learning: Emergency Rejections")
520 plt.xlabel("Episode")
521 plt.ylabel("Rejections")
522 plt.grid(True)
523
524 plt.subplot(1,3,3)
525 plt.plot(icu_q)
526 plt.title("Q-Learning: ICU Utilisation")
527 plt.xlabel("Episode")
528 plt.ylabel("Utilisation")
529 plt.grid(True)
530 plt.tight_layout()
531 plt.show()
532
533
534 # %% [markdown]
535 # ## Deep RL experiment runs
536 # This cell trains the configured deep RL variants for comparison.
537 #
538
539 # %%
540 # DQN variants
541 rewards_dqn = train_dqn(
542     episodes=1500,
543     double_dqn=False,
544     dueling=False
545 )
546
547 rewards_double = train_dqn(
548     episodes=1500,
549     double_dqn=True,

```

```

551     dueling=False
552 )
553
554 rewards_dueling = train_dqn(
555     episodes=1500,
556     double_dqn=True,
557     dueling=True
558 )
559
560 # Actor-Critic
561 rewards_ac, rejections_ac, icu_ac = train_actor_critic(episodes=1500)
562
563
564 # %% [markdown]
565 # ## Result plots
566 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
567 #
568
569 # %%
570 # 6. PLOTS: COMPARISONS
571
572 # Reward comparison across DRL algorithms
573 plt.figure(figsize=(10,5))
574 plt.plot(rewards_dqn, label="DQN")
575 plt.plot(rewards_double, label="Double DQN")
576 plt.plot(rewards_dueling, label="Dueling Double DQN")
577 plt.plot(rewards_ac, label="Actor-Critic")
578 plt.xlabel("Episode")
579 plt.ylabel("Total Reward")
580 plt.title("DRL Algorithms on Hospital Bed Allocation")
581 plt.legend()
582 plt.grid(True)
583 plt.show()
584
585
586 # %% [markdown]
587 # ## Result plots
588 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
589 #
590
591 # %%
592 # Actor-Critic detailed metrics
593 plt.figure(figsize=(15,4))
594
595 plt.subplot(1,3,1)
596 plt.plot(rewards_ac)
597 plt.title("Actor-Critic: Episode Reward")
598 plt.xlabel("Episode")
599 plt.ylabel("Reward")
600 plt.grid(True)
601
602 plt.subplot(1,3,2)
603 plt.plot(rejections_ac)
604 plt.title("Actor-Critic: Emergency Rejections")
605 plt.xlabel("Episode")
606 plt.ylabel("Rejections")
607 plt.grid(True)
608
609 plt.subplot(1,3,3)
610 plt.plot(icu_ac)
611 plt.title("Actor-Critic: ICU Utilisation")
612 plt.xlabel("Episode")
613 plt.ylabel("Utilisation")
614 plt.grid(True)
615
616 plt.tight_layout()
617 plt.show()

```

Listing 3: Actor-Critic Code

Advanced Task: Steps 8., and 9. - Proximal Policy Optimisation Actor-Critic Algorithm Yehoshua Contribution

```
1  # %% [markdown]
2  # # DRL Coursework 1b: PPO Actor-Critic Comparison
3  # This version keeps the old Part 1b bed-allocation environment and uses PPO as the actor-critic method.
4  #
5
6  # %% [markdown]
7  # ## Imports
8  # Libraries are grouped first so every later cell has the same dependencies available.
9  #
10
11 # %%
12 import random
13 from collections import deque
14 from itertools import product
15
16 import gymnasium as gym
17 import torch
18 import numpy as np
19 import pandas as pd
20 import matplotlib.pyplot as plt
21 import torch.nn as nn
22 import torch.optim as optim
23 from gymnasium import spaces
24
25 from stable_baselines3 import PPO
26 from stable_baselines3.common.monitor import Monitor
27
28 try:
29     from tqdm.auto import tqdm
30 except ImportError:
31     def tqdm(iterable, **kwargs):
32         return iterable
33
34 ACTION_MEANINGS = {
35     0: "allocate_icu",
36     1: "allocate_general",
37     2: "delay_patient",
38 }
39
40
41 # %% [markdown]
42 # ## Reproducibility setup
43 # A fixed seed makes random, NumPy, and PyTorch sampling easier to reproduce.
44 #
45
46 # %%
47 SEED = 42
48
49 random.seed(SEED)
50 np.random.seed(SEED)
51 torch.manual_seed(SEED)
52 if torch.cuda.is_available():
53     torch.cuda.manual_seed_all(SEED)
54
55 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
56 print(f"Using device: {device}")
57
58
59 # %% [markdown]
60 # ## Environment definition
61 # The hospital environment stores bed availability, patient type, actions, rewards, and transitions.
62 #
63
64 # %%
65 class HospitalEnv:
```

```

66     def __init__(self):
67         # ICU free: 0,1,2
68         # GEN free: 0,1,2,3
69         # Patient type: 0=emergency, 1=urgent, 2=elective
70         self.reset()
71
72     def reset(self):
73         self.icu_free = np.random.randint(0, 3)
74         self.gen_free = np.random.randint(0, 4)
75         self.ptype = np.random.choice([0,1,2])
76         return self._get_state()
77
78     def _get_state(self):
79         return (self.icu_free, self.gen_free, self.ptype)
80
81     def step(self, action):
82         reward = 0
83         icu, gen, p = self.icu_free, self.gen_free, self.ptype
84
85         # APPLY ACTION
86         if action == 0: # allocate ICU
87             if icu > 0:
88                 self.icu_free -= 1
89                 if p in [0,1]: reward += 5
90                 else: reward -= 3
91             else:
92                 reward -= 10
93
94         elif action == 1: # allocate general
95             if gen > 0:
96                 self.gen_free -= 1
97                 if p == 2: reward += 5
98                 elif p in [0,1]:
99                     if icu > 0: reward -= 3
100                     else: reward += 2
101             else:
102                 reward -= 10
103
104         elif action == 2: # delay
105             reward -= 1
106             if p == 0:
107                 reward -= 10
108
109         # DISCHARGES
110         if random.random() < 0.3:
111             self.icu_free = min(self.icu_free + 1, 2)
112         if random.random() < 0.4:
113             self.gen_free = min(self.gen_free + 1, 3)
114
115         # EMERGENCY REJECTION LOGIC
116         emergency_rejected = 0
117         if p == 0: # emergency patient
118             if action == 0 and icu == 0:
119                 emergency_rejected = 1
120             elif action == 1 and gen == 0:
121                 emergency_rejected = 1
122             elif action == 2:
123                 emergency_rejected = 1
124
125         # ICU utilisation: 1 if at least one ICU bed is occupied
126         icu_occupied = 1 if self.icu_free < 2 else 0
127
128         # NEW PATIENT ARRIVAL
129         self.ptype = np.random.choice([0,1,2], p=[0.2,0.3,0.5])
130
131         next_state = self._get_state()
132         done = False
133         return next_state, reward, done, emergency_rejected, icu_occupied
134
135

```

```

136 # %% [markdown]
137 # ## Tabular Q-learning
138 # These helpers map states to Q-table rows and update values with epsilon-greedy exploration.
139 #
140
141 # %%
142 def state_to_index(s):
143     icu, gen, p = s
144     return icu*12 + gen*3 + p # 36 states
145
146
147 # %% [markdown]
148 # ## Tabular Q-learning
149 # These helpers map states to Q-table rows and update values with epsilon-greedy exploration.
150 #
151
152 # %%
153 def run_q_learning(epochs=1000, alpha=0.1, gamma=0.9, epsilon=0.2):
154     env = HospitalEnv()
155     Q = np.zeros((36, 3))
156
157     rewards_per_episode = []
158     emergency_rejections = []
159     icu_utilisation = []
160
161     for ep in range(epochs):
162         s = env.reset()
163         total_reward = 0
164         episode_rejections = 0
165         episode_icu_occ = 0
166
167         for t in range(50):
168             s_idx = state_to_index(s)
169
170             # epsilon-greedy
171             if random.random() < epsilon:
172                 a = np.random.randint(0, 3)
173             else:
174                 a = np.argmax(Q[s_idx])
175
176             s_next, r, done, rej, icu_occ = env.step(a)
177             s_next_idx = state_to_index(s_next)
178
179             # Q update
180             Q[s_idx, a] += alpha * (r + gamma * np.max(Q[s_next_idx]) - Q[s_idx, a])
181
182             s = s_next
183             total_reward += r
184             episode_rejections += rej
185             episode_icu_occ += icu_occ
186
187             rewards_per_episode.append(total_reward)
188             emergency_rejections.append(episode_rejections)
189             icu_utilisation.append(episode_icu_occ / 50.0)
190
191         return Q, rewards_per_episode, emergency_rejections, icu_utilisation
192
193
194 # %% [markdown]
195 # ## Q-learning experiment
196 # This cell runs the tabular baseline and plots reward, rejection, or utilisation traces.
197 #
198
199 # %%
200 # 3. RUN EXPERIMENT + PLOTS
201
202 Q, rewards, rejections, icu_util = run_q_learning(epochs=1500)
203
204 plt.figure(figsize=(15,4))
205

```

```

206 plt.subplot(1,3,1)
207 plt.plot(rewards)
208 plt.title("Episode Reward")
209 plt.xlabel("Episode")
210 plt.ylabel("Reward")
211 plt.grid(True)
212
213 plt.subplot(1,3,2)
214 plt.plot(rejections)
215 plt.title("Emergency Rejections per Episode")
216 plt.xlabel("Episode")
217 plt.ylabel("Rejections")
218 plt.grid(True)
219
220 plt.subplot(1,3,3)
221 plt.plot(icu_util)
222 plt.title("ICU Utilisation Rate")
223 plt.xlabel("Episode")
224 plt.ylabel("Utilisation")
225 plt.grid(True)
226
227 plt.tight_layout()
228 plt.show()
229
230
231
232 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
233
234
235 # %% [markdown]
236 # ## State vector representation
237 # Deep RL models use numeric state vectors instead of integer Q-table indices.
238 #
239
240 # %%
241 def state_to_vector(s):
242     icu, gen, p = s
243     p_onehot = np.zeros(3)
244     p_onehot[p] = 1.0
245     return np.array([icu, gen] + p_onehot.tolist(), dtype=np.float32)
246
247
248 # %% [markdown]
249 # ## Replay buffer
250 # Experience replay stores transitions so DQN updates can sample mixed past experience.
251 #
252
253 # %%
254 class ReplayBuffer:
255     def __init__(self, capacity=10000):
256         self.buffer = deque(maxlen=capacity)
257
258     def push(self, state, action, reward, next_state, done):
259         self.buffer.append((state, action, reward, next_state, done))
260
261     def sample(self, batch_size):
262         batch = random.sample(self.buffer, batch_size)
263         states, actions, rewards, next_states, dones = zip(*batch)
264         return (
265             torch.tensor(states, dtype=torch.float32, device=device),
266             torch.tensor(actions, dtype=torch.long, device=device),
267             torch.tensor(rewards, dtype=torch.float32, device=device),
268             torch.tensor(next_states, dtype=torch.float32, device=device),
269             torch.tensor(dones, dtype=torch.float32, device=device),
270         )
271
272     def __len__(self):
273         return len(self.buffer)
274
275

```

```

276 # %% [markdown]
277 # ## DQN network models
278 # Standard and dueling networks estimate action values for each state.
279 #
280
281 # %%
282 class DQN(nn.Module):
283     def __init__(self, state_dim=5, action_dim=3, hidden_dim=64):
284         super().__init__()
285         self.net = nn.Sequential(
286             nn.Linear(state_dim, hidden_dim),
287             nn.ReLU(),
288             nn.Linear(hidden_dim, hidden_dim),
289             nn.ReLU(),
290             nn.Linear(hidden_dim, action_dim)
291         )
292
293     def forward(self, x):
294         return self.net(x)
295
296
297 # %% [markdown]
298 # ## DQN network models
299 # Standard and dueling networks estimate action values for each state.
300 #
301
302 # %%
303 class DuelingDQN(nn.Module):
304     def __init__(self, state_dim=5, action_dim=3, hidden_dim=64):
305         super().__init__()
306         self.feature = nn.Sequential(
307             nn.Linear(state_dim, hidden_dim),
308             nn.ReLU()
309         )
310         self.value_stream = nn.Sequential(
311             nn.Linear(hidden_dim, hidden_dim),
312             nn.ReLU(),
313             nn.Linear(hidden_dim, 1)
314         )
315         self.adv_stream = nn.Sequential(
316             nn.Linear(hidden_dim, hidden_dim),
317             nn.ReLU(),
318             nn.Linear(hidden_dim, action_dim)
319         )
320
321     def forward(self, x):
322         feat = self.feature(x)
323         value = self.value_stream(feat)
324         adv = self.adv_stream(feat)
325         return value + adv - adv.mean(dim=1, keepdim=True)
326
327
328 # %% [markdown]
329 # ## DQN training loop
330 # The training loop supports standard DQN, Double DQN, and Dueling Double DQN.
331 #
332
333 # %%
334 def train_dqn(
335     episodes=1000,
336     gamma=0.99,
337     lr=1e-3,
338     batch_size=64,
339     buffer_capacity=10000,
340     min_buffer_size=1000,
341     epsilon_start=1.0,
342     epsilon_end=0.05,
343     epsilon_decay=500,
344     target_update_freq=50,
345     double_dqn=True,

```

```

346     dueling=False,
347     icu_capacity=2
348 ):
349     env = HospitalEnv()
350     state_dim = 5
351     action_dim = 3
352
353     if dueling:
354         policy_net = DuelingDQN(state_dim, action_dim).to(device)
355         target_net = DuelingDQN(state_dim, action_dim).to(device)
356     else:
357         policy_net = DQN(state_dim, action_dim).to(device)
358         target_net = DQN(state_dim, action_dim).to(device)
359
360     target_net.load_state_dict(policy_net.state_dict())
361     target_net.eval()
362
363     optimizer = optim.Adam(policy_net.parameters(), lr=lr)
364     replay_buffer = ReplayBuffer(capacity=buffer_capacity)
365
366     rewards_per_episode = []
367     emergency_rej_per_episode = []
368     icu_util_per_episode = []
369
370     steps_done = 0
371
372     for ep in range(epochs):
373         s = env.reset()
374         s_vec = state_to_vector(s)
375         total_reward = 0
376
377         emergency_rej = 0
378         icu_util_sum = 0.0
379         step_count = 0
380
381         for t in range(50):
382             epsilon = epsilon_end + (epsilon_start - epsilon_end) * np.exp(-steps_done / epsilon_decay)
383             steps_done += 1
384
385             if random.random() < epsilon:
386                 a = np.random.randint(0, action_dim)
387             else:
388                 with torch.no_grad():
389                     q_values = policy_net(torch.tensor([s_vec], device=device))
390                     a = q_values.argmax(dim=1).item()
391
392             s_next, r, done, rej, icu_occ = env.step(a)
393
394             emergency_rej += rej
395             icu_util_sum += icu_occ
396             step_count += 1
397
398             s_next_vec = state_to_vector(s_next)
399
400             replay_buffer.push(s_vec, a, r, s_next_vec, done)
401
402             s_vec = s_next_vec
403             s = s_next
404             total_reward += r
405
406             if len(replay_buffer) >= min_buffer_size:
407                 states, actions, rewards, next_states, dones = replay_buffer.sample(batch_size)
408
409                 q_values = policy_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)
410
411                 with torch.no_grad():
412                     if double_dqn:
413                         next_actions = policy_net(next_states).argmax(dim=1, keepdim=True)
414                         next_q_target = target_net(next_states).gather(1, next_actions).squeeze(1)
415                     else:

```

```

416         next_q_target = target_net(next_states).max(dim=1)[0]
417
418         targets = rewards + gamma * (1 - dones) * next_q_target
419
420         loss = nn.MSELoss()(q_values, targets)
421         optimizer.zero_grad()
422         loss.backward()
423         optimizer.step()
424
425         if done:
426             break
427
428         rewards_per_episode.append(total_reward)
429         emergency_rej_per_episode.append(emergency_rej)
430         icu_util_per_episode.append(icu_util_sum / step_count)
431
432         if ep % target_update_freq == 0:
433             target_net.load_state_dict(policy_net.state_dict())
434
435     return rewards_per_episode, emergency_rej_per_episode, icu_util_per_episode
436
437
438 # %% [markdown]
439 # ## Actor-Critic networks
440 # Actor-Critic models learn a policy and a value estimate instead of only Q-values.
441 #
442
443 # %%
444 class Actor(nn.Module):
445     def __init__(self, state_dim=5, action_dim=3, hidden_dim=64):
446         super().__init__()
447         self.net = nn.Sequential(
448             nn.Linear(state_dim, hidden_dim),
449             nn.ReLU(),
450             nn.Linear(hidden_dim, hidden_dim),
451             nn.ReLU(),
452         )
453         self.policy_head = nn.Linear(hidden_dim, action_dim)
454
455     def forward(self, x):
456         logits = self.policy_head(self.net(x))
457         return torch.softmax(logits, dim=-1)
458
459
460 # %% [markdown]
461 # ## Actor-Critic networks
462 # Actor-Critic models learn a policy and a value estimate instead of only Q-values.
463 #
464
465 # %%
466 class Critic(nn.Module):
467     def __init__(self, state_dim=5, hidden_dim=64):
468         super().__init__()
469         self.net = nn.Sequential(
470             nn.Linear(state_dim, hidden_dim),
471             nn.ReLU(),
472             nn.Linear(hidden_dim, hidden_dim),
473             nn.ReLU(),
474             nn.Linear(hidden_dim, 1)
475         )
476
477     def forward(self, x):
478         return self.net(x).squeeze(-1)
479
480
481 # %% [markdown]
482 # ## Gymnasium Adapter for Notebook Environment
483 # This adapter uses the 'HospitalEnv' defined above so PPO does not rely on an external environment script.
484
485 # %%

```

```

486 class HospitalGymWrapper(gym.Env):
487     """Gymnasium adapter around the notebook original HospitalEnv."""
488
489     metadata = {"name": "HospitalGymWrapper"}
490
491     def __init__(self, max_steps=50):
492         super().__init__()
493         self.max_steps = int(max_steps)
494         self.env = HospitalEnv()
495         self.action_space = spaces.Discrete(3)
496         self.observation_space = spaces.Box(
497             low=np.asarray([0, 0, 0, 0, 0], dtype=np.float32),
498             high=np.asarray([2, 3, 2, 1, 1], dtype=np.float32),
499             dtype=np.float32,
500         )
501         self.current_state = None
502         self.timestep = 0
503
504     def reset(self, seed=None, options=None):
505         super().reset(seed=seed)
506         if seed is not None:
507             random.seed(seed)
508             np.random.seed(seed)
509         self.env = HospitalEnv()
510         self.current_state = self.env.reset()
511         self.timestep = 0
512         return self._get_observation(), self._get_info()
513
514     def step(self, action):
515         action = int(np.clip(action, 0, self.action_space.n - 1))
516         self.timestep += 1
517
518         next_state, reward, done, emergency_rejected, icu_occupied = self.env.step(action)
519         self.current_state = next_state
520
521         terminated = bool(done)
522         truncated = self.timestep >= self.max_steps
523         info = self._get_info(
524             action=action,
525             reward=reward,
526             emergency_rejected=emergency_rejected,
527             icu_occupied=icu_occupied,
528         )
529         return self._get_observation(), float(reward), terminated, truncated, info
530
531     def _get_observation(self):
532         return state_to_vector(self.current_state).astype(np.float32)
533
534     def _get_info(self, action=None, reward=0.0, emergency_rejected=0, icu_occupied=0):
535         icu_free, gen_free, patient_type = self.current_state
536         return {
537             "timestep": self.timestep,
538             "action": action,
539             "action_meaning": ACTION_MEANINGS.get(action) if action is not None else None,
540             "reward": float(reward),
541             "icu_free": int(icu_free),
542             "general_free": int(gen_free),
543             "patient_type": int(patient_type),
544             "emergency_rejection": int(emergency_rejected),
545             "icu_occupied": int(icu_occupied),
546             "icu_utilisation": float(icu_occupied),
547         }
548
549
550     # %% [markdown]
551     # ## PPO Actor-Critic Setup
552     # PPO is used here as the actor-critic method while keeping the old Part 1b environment logic.
553
554     # %%
555     learning_rates = [1e-4, 3e-4, 1e-3]

```

```

556 batch_sizes = [32, 64, 128]
557 n_steps_values = [128, 256, 512]
558
559 PPO_FIXED_PARAMS = {
560     "gamma": 0.99,
561     "gae_lambda": 0.95,
562     "clip_range": 0.2,
563     "ent_coef": 0.01,
564     "n_epochs": 25,
565 }
566
567
568 def make_ppo_old_env(max_steps=50):
569     return Monitor(HospitalGymWrapper(max_steps=max_steps))
570
571
572 def train_ppo(learning_rate, batch_size, n_steps, total_timesteps=30000, seed=42, max_steps=50):
573     # Train one PPO model for a single grid-search configuration.
574     train_env = make_ppo_old_env(max_steps=max_steps)
575
576     model = PPO(
577         policy="MlpPolicy",
578         env=train_env,
579         learning_rate=learning_rate,
580         n_steps=n_steps,
581         batch_size=batch_size,
582         n_epochs=PPO_FIXED_PARAMS["n_epochs"],
583         gamma=PPO_FIXED_PARAMS["gamma"],
584         gae_lambda=PPO_FIXED_PARAMS["gae_lambda"],
585         clip_range=PPO_FIXED_PARAMS["clip_range"],
586         ent_coef=PPO_FIXED_PARAMS["ent_coef"],
587         verbose=0,
588         seed=seed,
589         device=device,
590     )
591
592     model.learn(total_timesteps=total_timesteps)
593     return model
594
595
596 def train_ppo_actor_critic(total_timesteps=75000, seed=42, max_steps=50):
597     # Backwards-compatible helper using the best default candidate.
598     return train_ppo(1e-4, 64, 256, total_timesteps=total_timesteps, seed=seed, max_steps=max_steps)
599
600
601 # %% [markdown]
602 ## PPO Evaluation Helpers
603 # These helpers match the existing reward, emergency rejection, and ICU utilisation outputs.
604
605 # %%
606 def evaluate_ppo(model, episodes=100, seed=42, max_steps=50):
607     # Evaluate PPO using reward, safety, utilisation, delay, and action metrics.
608     env = HospitalGymWrapper(max_steps=max_steps)
609     rows = []
610     action_counts = {action: 0 for action in ACTION_MEANINGS}
611
612     for ep in range(episodes):
613         obs, info = env.reset(seed=seed + ep)
614         done = False
615         total_reward = 0.0
616         emergency_rejections = 0
617         icu_values = []
618         delayed_patients = 0
619         steps = 0
620
621         while not done:
622             action, _ = model.predict(obs, deterministic=True)
623             action = int(np.asarray(action).item())
624             action_counts[action] += 1
625

```

```

626         obs, reward, terminated, truncated, info = env.step(action)
627         done = terminated or truncated
628
629         total_reward += float(reward)
630         emergency_rejections += int(info.get("emergency_rejection", 0))
631         icu_values.append(float(info.get("icu_occupied", 0)))
632         delayed_patients += int(action == 2)
633         steps += 1
634
635         rows.append({
636             "episode": ep,
637             "total_reward": total_reward,
638             "emergency_rejections": emergency_rejections,
639             "icu_utilisation": float(np.mean(icu_values)) if icu_values else 0.0,
640             "delayed_patients": delayed_patients,
641             "steps": steps,
642         })
643
644     results_df = pd.DataFrame(rows)
645     action_counts_df = pd.DataFrame({
646         "action": list(action_counts.keys()),
647         "action_name": [ACTION_MEANINGS[action] for action in action_counts],
648         "count": list(action_counts.values()),
649     })
650
651     summary = {
652         "mean_reward": results_df["total_reward"].mean(),
653         "std_reward": results_df["total_reward"].std(),
654         "emergency_rejections": results_df["emergency_rejections"].mean(),
655         "icu_utilisation": results_df["icu_utilisation"].mean(),
656         "delayed_patients": results_df["delayed_patients"].mean(),
657     }
658     return summary, results_df, action_counts_df
659
660
661 def evaluate_ppo_actor_critic(model, episodes=1500, seed=42, max_steps=50):
662     # Backwards-compatible output format for the original comparison plots.
663     summary, results_df, action_counts_df = evaluate_ppo(model, episodes=episodes, seed=seed, max_steps=
max_steps)
664     return (
665         results_df["total_reward"].tolist(),
666         results_df["emergency_rejections"].tolist(),
667         results_df["icu_utilisation"].tolist(),
668         action_counts_df,
669     )
670
671
672 def evaluate_random_old_env(episodes=1500, seed=42, max_steps=50):
673     rng = np.random.default_rng(seed)
674     env = HospitalGymWrapper(max_steps=max_steps)
675     rewards_per_episode = []
676     emergency_rej_per_episode = []
677     icu_util_per_episode = []
678
679     for ep in range(episodes):
680         obs, info = env.reset(seed=seed + ep)
681         done = False
682         total_reward = 0.0
683         emergency_rej = 0
684         icu_util_sum = 0.0
685         step_count = 0
686
687         while not done:
688             action = int(rng.integers(env.action_space.n))
689             obs, reward, terminated, truncated, info = env.step(action)
690             done = terminated or truncated
691
692             total_reward += float(reward)
693             emergency_rej += int(info.get("emergency_rejection", 0))
694             icu_util_sum += float(info.get("icu_occupied", 0))

```

```

695         step_count += 1
696
697         rewards_per_episode.append(total_reward)
698         emergency_rej_per_episode.append(emergency_rej)
699         icu_util_per_episode.append(icu_util_sum / step_count if step_count else 0.0)
700
701     return rewards_per_episode, emergency_rej_per_episode, icu_util_per_episode
702
703
704     # %% [markdown]
705     # ## PPO Grid Search Explanation
706     # Grid search trains every specified parameter combination and ranks them by validation metrics.
707
708     # %% [markdown]
709     # The tuned parameters control update size and rollout batching; 'gamma' and 'gae_lambda' are kept fixed to
710     # preserve the reward horizon and advantage estimator.
711
712     # %%
713     def valid_grid_combinations(learning_rates, batch_sizes, n_steps_values):
714         # Keep rollout batches compatible with Stable-Baselines3 PPO.
715         for learning_rate, batch_size, n_steps in product(learning_rates, batch_sizes, n_steps_values):
716             if batch_size <= n_steps and n_steps % batch_size == 0:
717                 yield learning_rate, batch_size, n_steps
718
719     def run_grid_search(
720         total_timesteps=30000,
721         eval_episodes=100,
722         seed=42,
723         max_steps=50,
724         output_csv="ppo_1b_grid_search_results.csv",
725     ):
726         # Train and evaluate every PPO hyperparameter combination.
727         rows = []
728         models = {}
729         action_count_frames = {}
730         combinations = list(valid_grid_combinations(learning_rates, batch_sizes, n_steps_values))
731
732         for idx, (learning_rate, batch_size, n_steps) in enumerate(tqdm(combinations, desc="PPO grid search")):
733             run_seed = seed + idx
734             model = train_ppo(
735                 learning_rate=learning_rate,
736                 batch_size=batch_size,
737                 n_steps=n_steps,
738                 total_timesteps=total_timesteps,
739                 seed=run_seed,
740                 max_steps=max_steps,
741             )
742             summary, episode_df, action_counts_df = evaluate_ppo(
743                 model,
744                 episodes=eval_episodes,
745                 seed=run_seed,
746                 max_steps=max_steps,
747             )
748
749             combo_label = f"lr={learning_rate:g}, batch={batch_size}, steps={n_steps}"
750             rows.append({
751                 "combination": combo_label,
752                 "learning_rate": learning_rate,
753                 "batch_size": batch_size,
754                 "n_steps": n_steps,
755                 "seed": run_seed,
756                 **summary,
757             })
758             models[combo_label] = model
759             action_count_frames[combo_label] = action_counts_df
760
761         results_df = pd.DataFrame(rows).sort_values(
762             by=["mean_reward", "emergency_rejections"],

```

```

763         ascending=[False, True],
764     ).reset_index(drop=True)
765     results_df["rank"] = np.arange(1, len(results_df) + 1)
766     results_df.to_csv(output_csv, index=False)
767
768     best_row = results_df.iloc[0]
769     best_label = best_row["combination"]
770     print("Best PPO configuration:")
771     print(best_row[["learning_rate", "batch_size", "n_steps", "mean_reward", "emergency_rejections"]])
772
773     return results_df, models[best_label], best_row.to_dict(), action_count_frames[best_label]
774
775
776     # %% [markdown]
777     # ## Grid Search Plotting
778     # These report-ready figures compare reward and emergency rejection outcomes across combinations.
779
780     # %%
781     def plot_results(grid_results_df):
782         ordered = grid_results_df.sort_values("rank")
783
784         plt.figure(figsize=(12, 5))
785         plt.bar(ordered["combination"], ordered["mean_reward"], yerr=ordered["std_reward"], capsize=3)
786         plt.xticks(rotation=45, ha="right")
787         plt.ylabel("Mean episode reward")
788         plt.title("PPO Grid Search: Mean Reward by Hyperparameter Combination")
789         plt.tight_layout()
790         plt.show()
791
792         plt.figure(figsize=(12, 5))
793         plt.bar(ordered["combination"], ordered["emergency_rejections"])
794         plt.xticks(rotation=45, ha="right")
795         plt.ylabel("Mean emergency rejections")
796         plt.title("PPO Grid Search: Emergency Rejections by Hyperparameter Combination")
797         plt.tight_layout()
798         plt.show()
799
800
801     # %% [markdown]
802     # ## Actor-Critic training loop
803     # This loop updates policy and value losses from simulated hospital episodes.
804     #
805
806     # %%
807     rewards_dqn, emrej_dqn, icu_dqn = train_dqn(
808         episodes=1500,
809         double_dqn=False,
810         dueling=False
811     )
812
813     rewards_double, emrej_double, icu_double = train_dqn(
814         episodes=1500,
815         double_dqn=True,
816         dueling=False
817     )
818
819     rewards_dueling, emrej_dueling, icu_dueling = train_dqn(
820         episodes=1500,
821         double_dqn=True,
822         dueling=True
823     )
824
825     grid_results_df, ppo_model, best_ppo_params, ppo_action_counts_df = run_grid_search(
826         total_timesteps=30000,
827         eval_episodes=100,
828         seed=SEED,
829         max_steps=50,
830     )
831
832     plot_results(grid_results_df)

```

```

833
834 rewards_ppo, emrej_ppo, icu_ppo, ppo_action_counts_df = evaluate_ppo_actor_critic(
835     ppo_model,
836     episodes=1500,
837     seed=SEED,
838     max_steps=50,
839 )
840
841 rewards_random, emrej_random, icu_random = evaluate_random_old_env(
842     episodes=1500,
843     seed=SEED,
844     max_steps=50,
845 )
846
847
848 # %% [markdown]
849 # ## Result plots
850 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
851 #
852
853 # %%
854 plt.figure(figsize=(10,5))
855 plt.plot(rewards_dueling, label="Dueling Double DQN")
856 plt.plot(rewards_ppo, label="PPO Actor-Critic", linewidth=2.5)
857 plt.xlabel("Episode")
858 plt.ylabel("Total Reward")
859 plt.title("Reward Comparison")
860 plt.legend()
861 plt.grid(True)
862 plt.show()
863
864
865 # %% [markdown]
866 # ## Result plots
867 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
868 #
869
870 # %%
871 plt.figure(figsize=(10,5))
872 plt.plot(emrej_dueling, label="Dueling Double DQN")
873 plt.plot(emrej_ppo, label="PPO Actor-Critic", linewidth=2.5)
874 plt.xlabel("Episode")
875 plt.ylabel("Emergency Rejections")
876 plt.title("Emergency Rejections per Episode")
877 plt.legend()
878 plt.grid(True)
879 plt.show()
880
881
882 # %% [markdown]
883 # ## Result plots
884 # Plots compare reward, emergency rejection, and ICU utilisation trends across methods.
885 #
886
887 # %%
888 plt.figure(figsize=(10,5))
889 plt.plot(icu_dqn, label="DQN")
890 plt.plot(icu_double, label="Double DQN")
891 plt.plot(icu_dueling, label="Dueling Double DQN")
892 plt.plot(icu_ppo, label="PPO Actor-Critic", linewidth=2.5)
893 plt.xlabel("Episode")
894 plt.ylabel("ICU Utilisation")
895 plt.title("ICU Utilisation per Episode")
896 plt.legend()
897 plt.grid(True)
898 plt.show()
899
900
901 # %% [markdown]
902 # ## PPO Results Table

```

```

903 # This converts the PPO and random baseline curves into Part C style summary tables.
904
905 # %%
906 ppo_1b_results = pd.DataFrame({
907     "episode": np.arange(len(rewards_ppo)),
908     "total_reward": rewards_ppo,
909     "emergency_rejections": emrej_ppo,
910     "icu_utilisation": icu_ppo,
911 })
912
913 random_1b_results = pd.DataFrame({
914     "episode": np.arange(len(rewards_random)),
915     "total_reward": rewards_random,
916     "emergency_rejections": emrej_random,
917     "icu_utilisation": icu_random,
918 })
919
920 ppo_1b_summary = pd.DataFrame([
921     {
922         "metric": metric,
923         "ppo_mean": ppo_1b_results[metric].mean(),
924         "ppo_std": ppo_1b_results[metric].std(),
925         "random_mean": random_1b_results[metric].mean(),
926         "random_std": random_1b_results[metric].std(),
927     }
928     for metric in ["total_reward", "emergency_rejections", "icu_utilisation"]
929 ])
930
931 print("Best PPO hyperparameters:")
932 print(pd.Series(best_ppo_params))
933 ppo_1b_summary
934
935
936 # %% [markdown]
937 # ## PPO versus Random Baseline
938 # These plots give the Part C style PPO/random comparison for the old Part 1b environment.
939
940 # %%
941 fig, axes = plt.subplots(1, 3, figsize=(16, 4))
942
943 axes[0].plot(ppo_1b_results["episode"], ppo_1b_results["total_reward"], label="PPO")
944 axes[0].plot(random_1b_results["episode"], random_1b_results["total_reward"], label="Random", alpha=0.7)
945 axes[0].set_title("Episode Reward")
946 axes[0].set_xlabel("Episode")
947 axes[0].legend()
948
949 axes[1].plot(ppo_1b_results["episode"], ppo_1b_results["emergency_rejections"], label="PPO")
950 axes[1].plot(random_1b_results["episode"], random_1b_results["emergency_rejections"], label="Random",
951             alpha=0.7)
952 axes[1].set_title("Emergency Rejections")
953 axes[1].set_xlabel("Episode")
954 axes[1].legend()
955
956 axes[2].plot(ppo_1b_results["episode"], ppo_1b_results["icu_utilisation"], label="PPO")
957 axes[2].plot(random_1b_results["episode"], random_1b_results["icu_utilisation"], label="Random", alpha
958             =0.7)
959 axes[2].set_title("ICU Utilisation")
960 axes[2].set_xlabel("Episode")
961 axes[2].legend()
962
963 plt.tight_layout()
964 plt.show()
965
966 # %% [markdown]
967 # ## PPO Action Frequencies
968 # This checks whether PPO mainly allocates ICU, allocates general beds, or delays patients.
969
970 # %%
971 plt.figure(figsize=(6, 4))

```

```

971 plt.bar(ppo_action_counts_df["action_name"], ppo_action_counts_df["count"])
972 plt.title("PPO Actor-Critic Action Frequencies")
973 plt.ylabel("Count")
974 plt.tight_layout()
975 plt.show()
976
977 ppo_action_counts_df
978
979
980 # %% [markdown]
981 # ## Save PPO 1b Outputs
982 # These files use 1b-specific names so they do not overwrite Part C PPO outputs.
983
984 # %%
985 ppo_model.save("ppo_1b_actor_critic_model")
986 ppo_1b_results.to_csv("ppo_1b_actor_critic_results.csv", index=False)
987 random_1b_results.to_csv("random_1b_baseline_results.csv", index=False)
988 ppo_1b_summary.to_csv("ppo_1b_actor_critic_summary.csv", index=False)
989 ppo_action_counts_df.to_csv("ppo_1b_action_counts.csv", index=False)
990 grid_results_df.to_csv("ppo_1b_grid_search_results.csv", index=False)
991
992 print("Saved PPO 1b model, grid search, and result CSV files.")

```

Listing 4: PPO Actor-Critic Code